



A Higher Purpose

Design Specification for the F-17 Humanitarian Hummingbird

Date: April 7, 2026

Course Name: ENEL 400 - Electrical Engineering Design and Technical Communications

Team Name: Specific Dynamics - Hood Research Department

Team Number: Group 16

Members: Khai Huynh, Vikram Procter, Minh Do, Caleb Garcia, Chloe Fulbrook, Ethan Sam

Instructor: Dr. Yani Jazayeri

TA: Benjamin Pele

Table of Contents

Acknowledgements	4
1. Introduction	4
1.1. Acronyms and Abbreviations	5
2. Predecessor Works and AI Tool Use	6
2.1. Predecessor Works	6
2.2. AI Tool Use	6
3. System Block Diagram	7
3.1. Hardware Block Diagram	7
3.1.1. High-Level System Block Diagram	8
3.1.2 ESC / Airframe Board Block Diagram	9
3.1.3 Avionics / Flight Controller Board Block Diagram	11
3.1.4 Handheld Controller Block Diagram	12
3.2. Firmware Block Diagram	14
3.2.1 ESC Firmware Block Diagram	15
3.2.2 Flight Controller Firmware Block Diagram	15
4. Block Descriptions	17
4.1. Hardware Blocks	17
4.1.1.	18
4.2. Firmware Blocks	18
4.2.1. Remote Control Firmware Blocks	18
4.2.1.1. Transmitter Firmware Blocks	18
4.2.1.1.1. System Initialization	18
4.2.1.1.2. Input Acquisition	18
4.2.1.1.3. Signal Processing	19
4.2.1.1.4. Calibration Subsystem	19
4.2.1.1.5. Control and Packetization	20
4.2.1.1.6. Communication and Transmission	20
4.2.1.1.7. User Feedback and OLED	20
4.2.1.1.8. System State Management	20
4.2.1.1.9. RTOS Scheduling	21
4.2.1.2. Receiver Firmware Blocks	21
4.2.1.2.1. System Initialization	21
4.2.1.2.2. ESP-NOW Receive Callback	22
4.2.1.2.3. RX Task	22
4.2.1.2.4. Controller Task	22
4.2.1.2.5. Stats Task	23
4.2.1.2.6. Shared State and Concurrency	23
4.2.1.2.7. RTOS Scheduling	24

4.2.2. PID Software Block	24
4.2.3. Initialization	24
4.2.4. Main PID Task	24
4.2.5. ESC Software Blocks	25
5. Engineering Analysis	27
5.1. Thrust and Weight Budget	27
5.1.1. Mass Budget	27
5.2. PWM Motor Mapping	28
5.3. PID Tuning - Ziegler-Nichols INspired Heuristic	28
5.4. Space Considerations	29
6. Enclosure	30
6.1 Overall Structural Stackup	30
6.2 Drone Landing Legs	30
6.3 Propeller Guards	31
6.4 Handheld Controller Enclosure	31
6.5 Design Iterations	31
7. Printed Circuit Boards	32
7.1 Custom BLDC ESC / Airframe PCB	32
7.2 Custom Avionics / Flight Controller PCB	33
7.3 Custom Handheld Controller PCB	33
8. Regulatory Codes	34
9. Design Alternatives Considered	36
9.1. Control Strategy: Cascaded PID vs. LQR vs. MPC	36
9.1. ESC: Custom Design vs. Commercial Off-The-Shelf	37
9.2. ESC MCU: STM32 vs. ESP32	38
9.3. Commutation Strategy: BEMF Sensorless vs. Hall-Sensored	38
9.4. Flight Controller: Custom vs. Commercial (Betaflight/INAV)	38
9.5. Wireless Protocol: ESP-NOW vs. CRSF/SBUS/Custom RF	39
9.6. Airframe: PCB-as-Frame vs. Traditional Frame	39
10. Future Work	40
10.1. UWB Indoor Positioning Reintegration	40
10.2. Payload Module	40
10.3. Regulatory Compliance Path	40
10.4. Manufacturing and Cost Optimization	40
10.5. Autonomous Flight	40
11. References	41
12. Appendix I: Schematic Diagrams	43
12.1. Electronic Speed Controller Board Schematic	43
12.2. Flight Controller Board Schematic	46

12.3. Remote Control Board Schematic	48
13. Appendix II: PCB Layouts	49
13.1. Electronic Speed Controller Board Layout	49
13.2. Flight Controller Board Layout	51
13.3. Remote Control Board Layout	53
14. Appendix III: Bill of Materials	54
15. Appendix IV: Prototype Photograph	63
16. Appendix V: 3D Model Drawings	65
16.1. Drone Landing Leg Drawing	65
16.2. Propeller Guards Drawing	66
16.3. Remote Controller Case Drawing	67
17. Appendix VI: Schedule and Expenditure Variance	67

Acknowledgements

Specific Dynamics would like to acknowledge Drs. Yani Jazayeri and Denis Onen, teaching assistants Ben Pele and Devin Atkin, and the ENEL 400 lab technicians for their support. We would also like to extend our thanks to Gauthier Appaix for his valuable contributions to our 3D designs and mechanical systems.

1. Introduction

This document presents the Design Specification for the F-17 Humanitarian Hummingbird microdrone, developed by the Hood Research Department of Specific Dynamics. It outlines prior works and includes an AI use disclosure, followed by system block diagrams, component descriptions, and engineering analysis. The report also details the final enclosure designs for both the drone and its remote controller, along with relevant regulatory codes and evaluated design alternatives. It concludes with recommendations for future work to further enhance the system. Supporting materials are provided in six appendices, including PCB schematics and layouts, a bill of materials, prototype images, 3D drawings, and a discussion of deviations from the original project schedule and budget.

The goal of the project was to create a modular, lightweight, cost-effective microdrone as a proof of concept for general-purpose UAV applications. In particular, the design addresses ongoing challenges in the UAV and remote sensing fields, where high costs and long development timelines often limit the feasibility of lightweight, application-specific drone solutions.

1.1. Acronyms and Abbreviations

This section provides an alphabetical list of acronyms and abbreviations used throughout this document.

- ADC — Analog-to-Digital Converter
- AGL — Above Ground Level
- AI — Artificial Intelligence
- BEMF — Back Electromotive Force
- BLDC — Brushless Direct Current
- BOM — Bill of Materials
- CAR — Canadian Aviation Regulations
- COTS — Commercial Off-The-Shelf
- CRSF — Crossfire (RC protocol)
- CSA — Canadian Standards Association
- DTS — Digital Transmission System

EIRP — Effective Isotropic Radiated Power
 EMC — Electromagnetic Compatibility
 ESC — Electronic Speed Controller
 ESP-IDF — Espressif IoT Development Framework
 FCC — Federal Communications Commission
 FHS — Frequency Hopping System
 GPIO — General-Purpose Input/Output
 HAL — Hardware Abstraction Layer
 I²C — Inter-Integrated Circuit
 IATA — International Air Transport Association
 IC — Integrated Circuit
 ICES — Interference-Causing Equipment Standard
 IEC — International Electrotechnical Commission
 IIR — Infinite Impulse Response
 IMU — Inertial Measurement Unit
 ISED — Innovation, Science and Economic Development Canada
 ISM — Industrial, Scientific, and Medical (radio band)
 JST — Japan Solderless Terminal (connector standard)
 LE-LAN — Licence-Exempt Local Area Network
 LiPo — Lithium Polymer (battery)
 MAC — Media Access Control
 MCU — Microcontroller Unit
 MEMS — Micro-Electro-Mechanical Systems
 MOSFET — Metal-Oxide-Semiconductor Field-Effect Transistor
 MVP — Minimum Viable Product
 NVS — Non-Volatile Storage
 OLED — Organic Light-Emitting Diode
 PCB — Printed Circuit Board
 PCBA — Printed Circuit Board Assembly
 PEP — Project Execution Plan
 PID — Proportional-Integral-Derivative
 PLA — Polylactic Acid
 PPM — Pulse Position Modulation
 PWM — Pulse Width Modulation
 RC — Radio Control
 RF — Radio Frequency
 RoHS — Restriction of Hazardous Substances
 RPAS — Remotely Piloted Aircraft System
 RSS — Radio Standards Specification
 RSSI — Received Signal Strength Indicator

SAR — Specific Absorption Rate
SBUS — Serial Bus (RC protocol)
SPI — Serial Peripheral Interface
TPU — Thermoplastic Polyurethane
UART — Universal Asynchronous Receiver-Transmitter
UAV — Unmanned Aerial Vehicle
USB — Universal Serial Bus
UWB — Ultra-Wideband
XT30 — 30 A current-rated battery connector standard

2. Predecessor Works and AI Tool Use

2.1. Predecessor Works

No direct predecessor works or prior team designs were used as a starting point for the F-17 Humanitarian Hummingbird; the project was developed from scratch. The team drew on publicly available reference materials during development, including the STMicroelectronics STSW-SPIN3204 6-step sensorless motor control reference library [10] (referenced in Section 9.2), the Espressif ESP-IDF documentation and example projects [13], the STM32 HAL reference manual [11], and component datasheets for all major ICs [12], [14], [15], [17]–[21].

2.2. AI Tool Use

The team used Anthropic's Claude as the primary AI tool throughout the project. Two interfaces were used: the Claude.ai web chat interface for conversational queries, and the Claude Code command-line interface for code-related tasks within the development environment. All six team members used these tools to varying degrees during the project.

AI assistance was used for firmware debugging, code comprehension, code review, and technical research. During firmware development, team members used Claude to diagnose bugs in the ESC commutation logic and PID control code, to interpret unfamiliar library APIs and hardware register configurations in the STM32 HAL and ESP-IDF frameworks, and to review existing code for correctness and potential issues. For research and learning, team members used Claude to build understanding of technical concepts encountered during the design process, such as sensorless BEMF commutation strategies, Mahony complementary filter implementation [22], cascaded PID control architectures, and datasheet interpretation for components including the

DRV8329 gate driver and ICM-40609-D IMU. AI tools were not used to generate firmware modules from scratch; all source code was written by team members.

All AI-generated output was manually reviewed and verified before inclusion in any deliverable. For firmware-related queries, team members cross-checked AI responses against component datasheets, reference manuals, and the actual behavior observed during bench testing. For report content, AI-assisted drafts were reviewed for technical accuracy against the team's design files and test results, and edited to reflect the project's actual implementation. The team found AI assistance to be generally reliable for the tasks described above, with no significant instances of incorrect information making it into final deliverables.

3. System Block Diagram

This section of the report demonstrates the overall system architecture through block diagrams. These provide a visual representation of how blocks in the system interact with each other, including both hardware and software architectures.

3.1. Hardware Block Diagram

The F-17 Humanitarian Hummingbird hardware is organized as a two-board stacked drone system with a separate handheld controller communicating wirelessly. Four hardware block diagrams are presented: a high-level system diagram showing how the three custom PCBs interconnect, followed by detailed diagrams of the ESC/airframe board, the avionics/flight controller board, and the handheld controller.

3.1.1. High-Level System Block Diagram

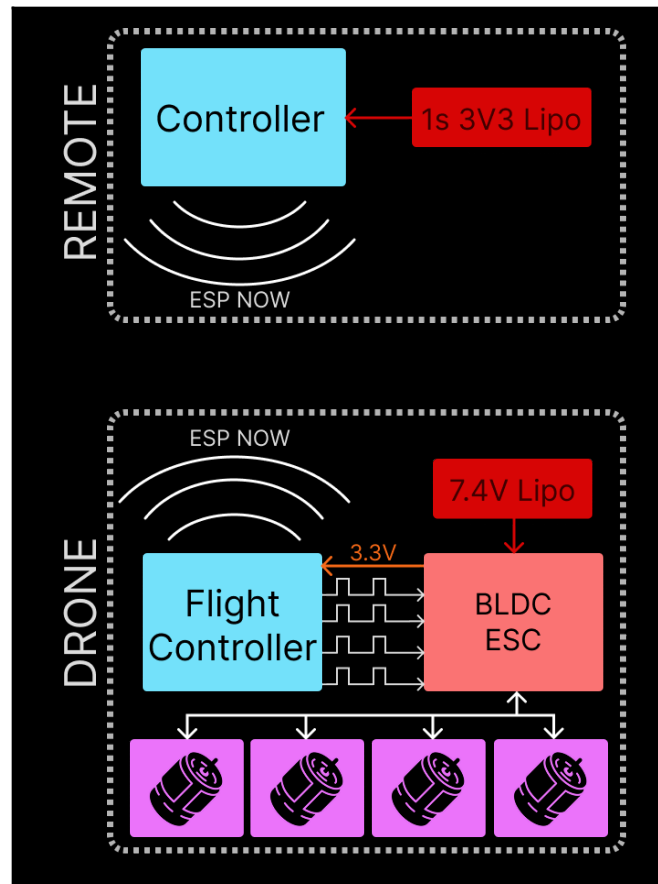


Figure 1: High level system architecture

The high-level system architecture consists of three custom PCBs and a wireless communication link. The bottom board in the drone stack is the custom ESC/airframe PCB, which serves as both the electronic speed controller and the primary structural airframe. It receives power from a 2S LiPo battery (7.4 V nominal) via an XT30 connector, drives the four YSIDO 1104 8600 kV brushless motors mounted at the board corners, and provides a regulated 3.3 V rail to the avionics board above. The top board is the custom avionics/flight controller PCB, which stacks above the ESC board via soldered wire connections. It runs the PID flight stabilization loop, reads the onboard inertial sensors, receives pilot commands over the ESP-NOW wireless link, and outputs four PWM motor commands to the ESC board below.

The handheld controller is a physically separate unit that reads pilot joystick inputs and transmits them to the drone over ESP-NOW powered by an independent 1S LiPo battery with USB-C charging. The ESP-NOW protocol provides a connectionless peer-to-peer data transport layer operating on the 2.4 GHz Wi-Fi physical layer,

requiring no router or access point. The controller and drone are paired using hardcoded MAC addresses on Wi-Fi channel 1.

Power originates from a GAONENG Flight 2S LiPo battery (1000 mAh, 120C) connected to the ESC board via an XT30 connector. The raw 7.4 V bus feeds directly to the four three-phase half-bridge motor drive stages. A TPS563209DDCR buck converter [19] on the ESC board steps the battery voltage down to 3.3 V for the STM32F405 MCU and gate driver logic. The avionics board receives 3.3 V from the ESC board through the board-to-board wiring, powering the ESP32-S3, IMU, and magnetometer. The handheld controller runs from its own independent 1S LiPo with an onboard charge management IC and USB-C charging port.

The overall signal path flows from the pilot's joystick inputs through the controller's ADC and filtering, across the ESP-NOW wireless link to the drone's ESP32-S3, into the cascaded PID control loop (where the IMU and magnetometer close the feedback path) and out as four PWM motor commands to the STM32F405 on the ESC board through the 6-step commutation logic, and into the four brushless motors.

3.1.2 ESC / Airframe Board Block Diagram

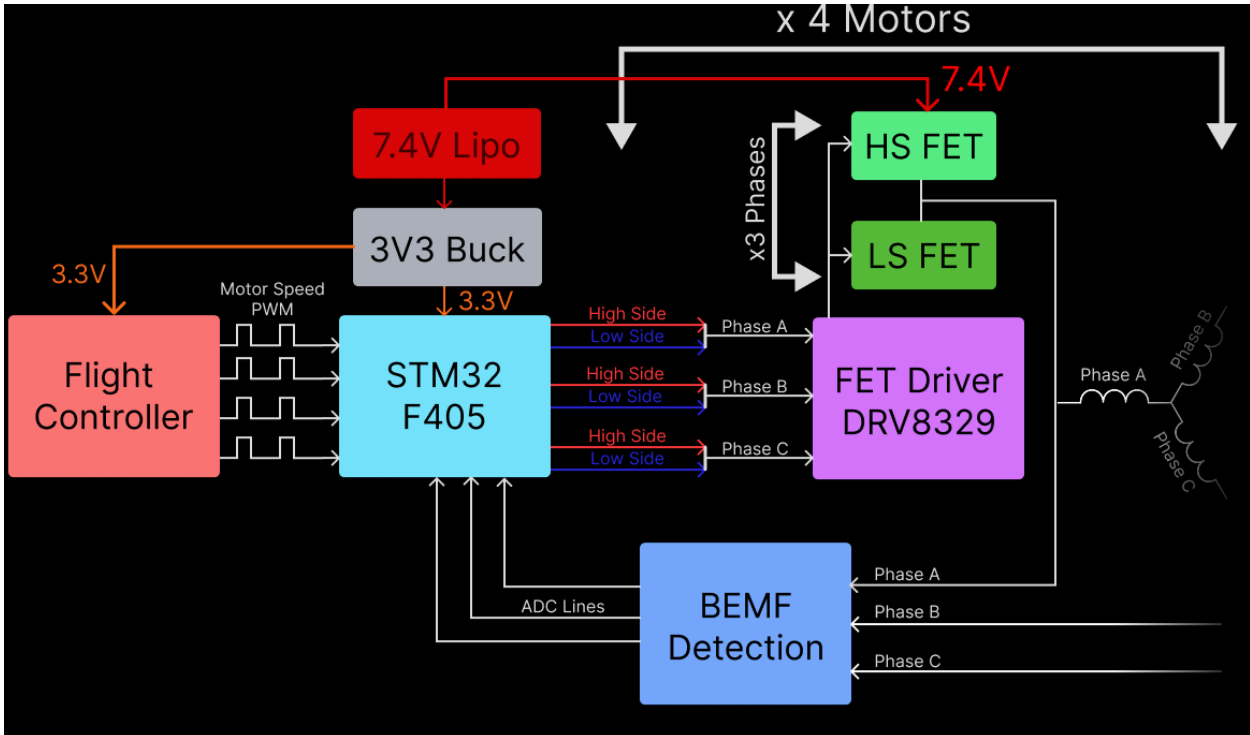


Figure 2: ESC / Airframe Board Block Diagram

3.1.3 Avionics / Flight Controller Board Block Diagram

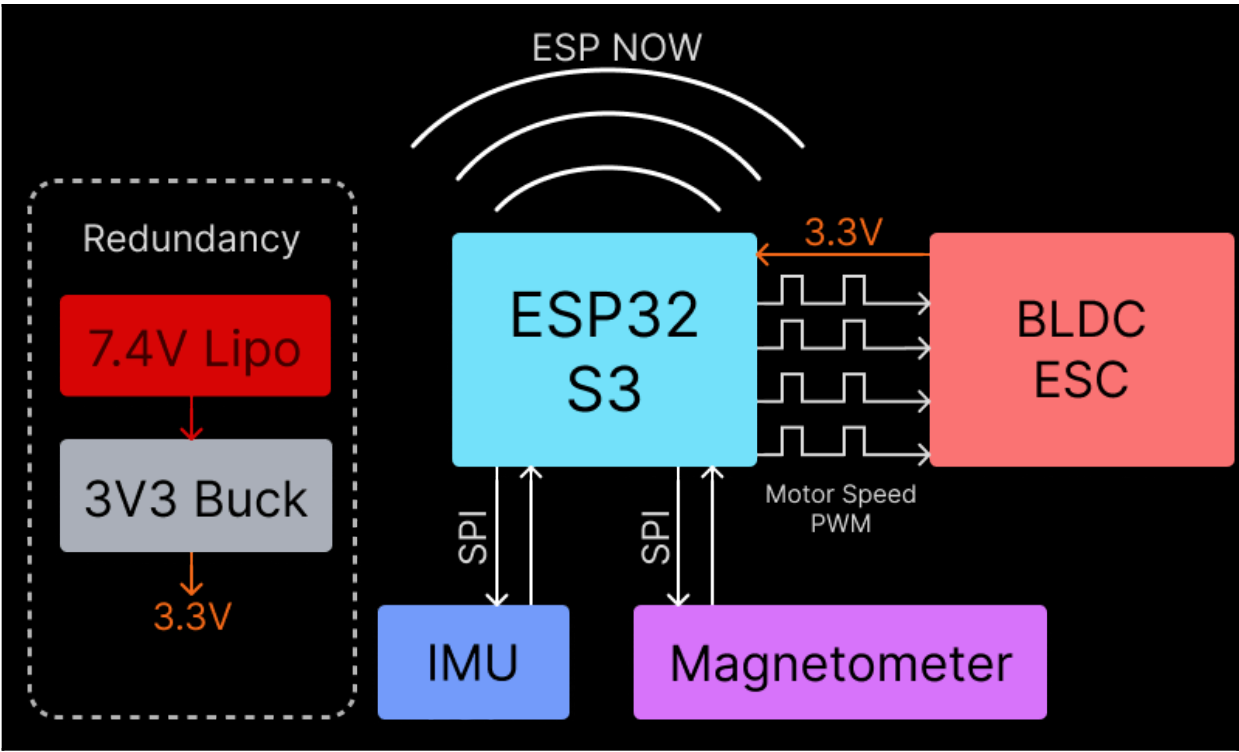


Figure 3: Avionics / Flight Controller Board Block Diagram

3.1.4 Handheld Controller Block Diagram

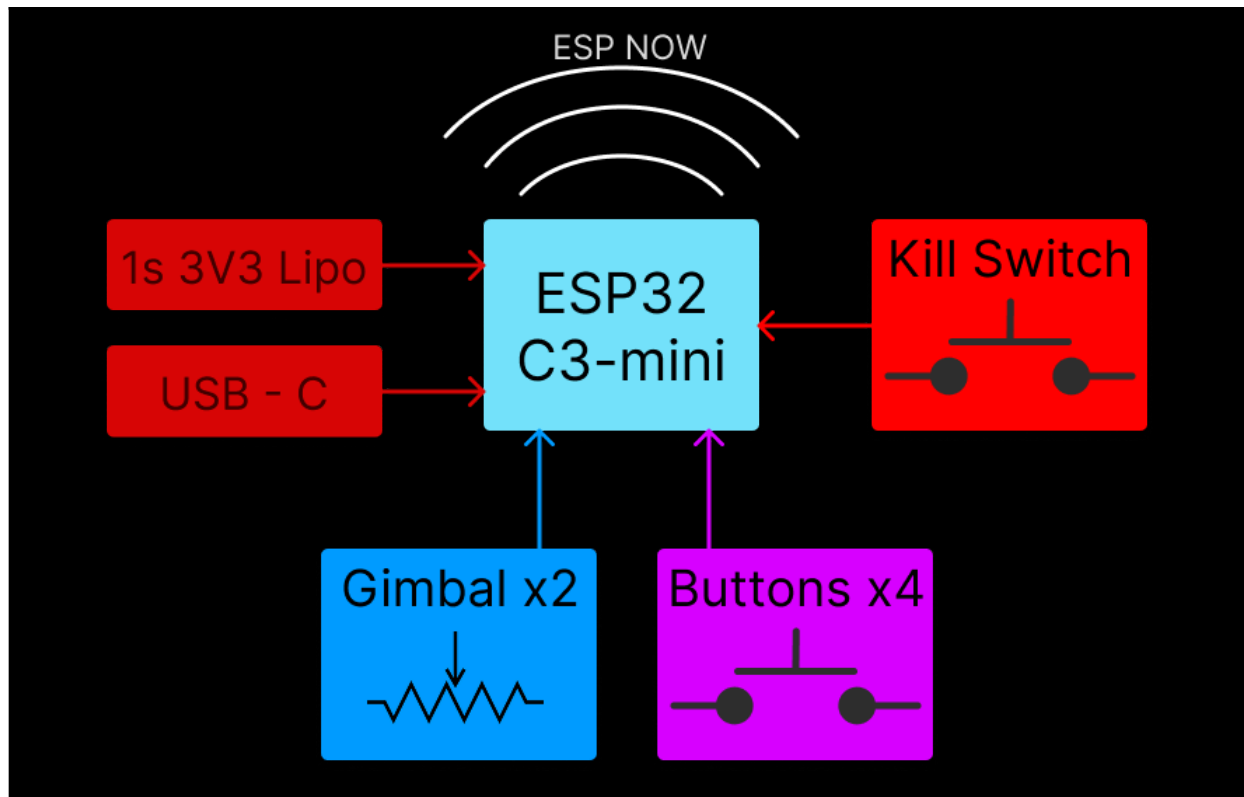


Figure 4: Controller Block Diagram

3.2. Firmware Block Diagram

Each PCB of the drone or controller has a microcontroller with custom C firmware. Much like the previous section each hardware element is mirrored by a firmware block diagram explaining the abstract operation of the MCUs on each board. In the following section an in-depth description for each firmware block diagram is provided.

3.2.1 ESC Firmware Block Diagram

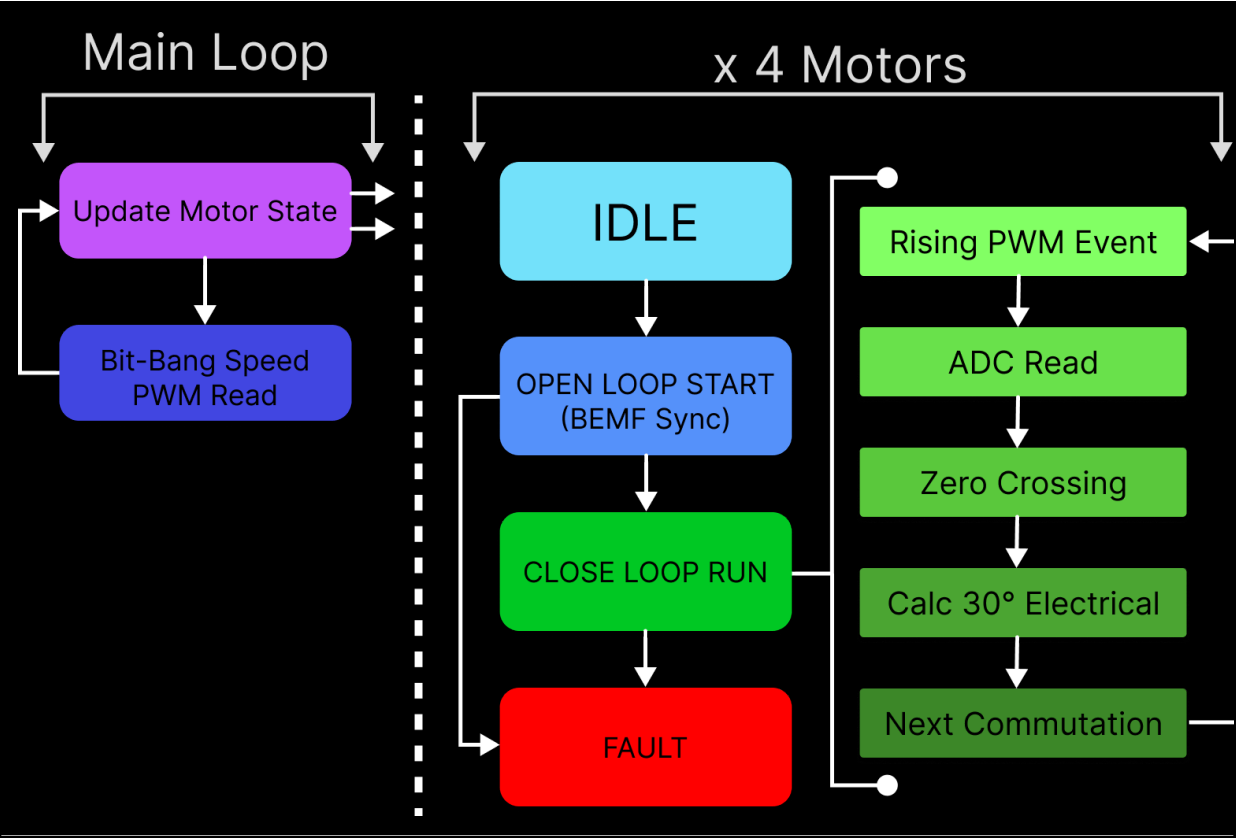


Figure 5: ESC Firmware Block Diagram

3.2.2 Flight Controller Firmware Block Diagram

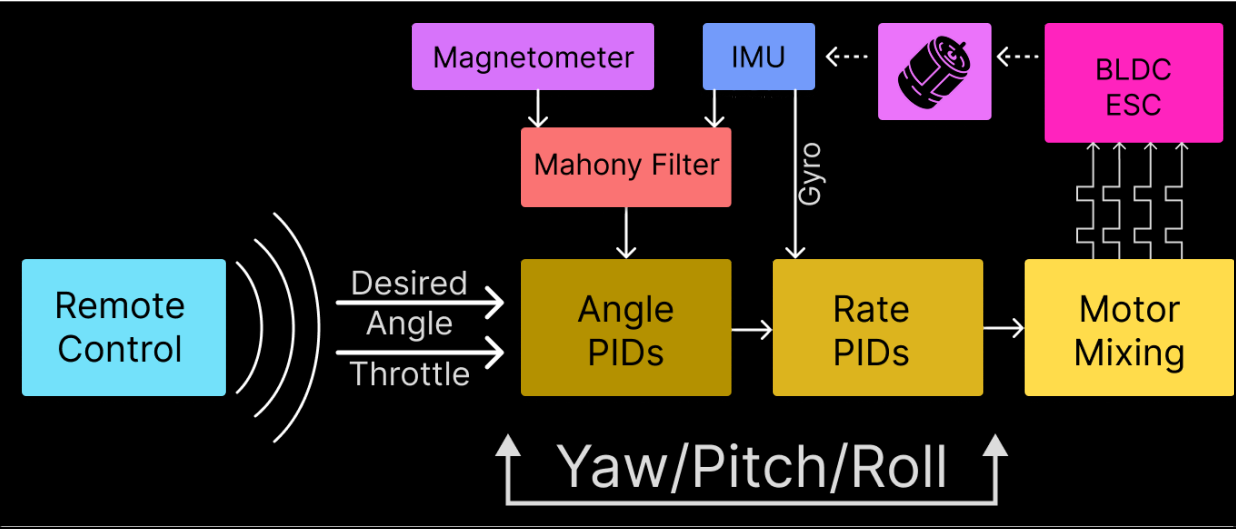


Figure 6: Controller Firmware Block Diagram

3.2.3 Flight Controller Receiver Firmware Block Diagram

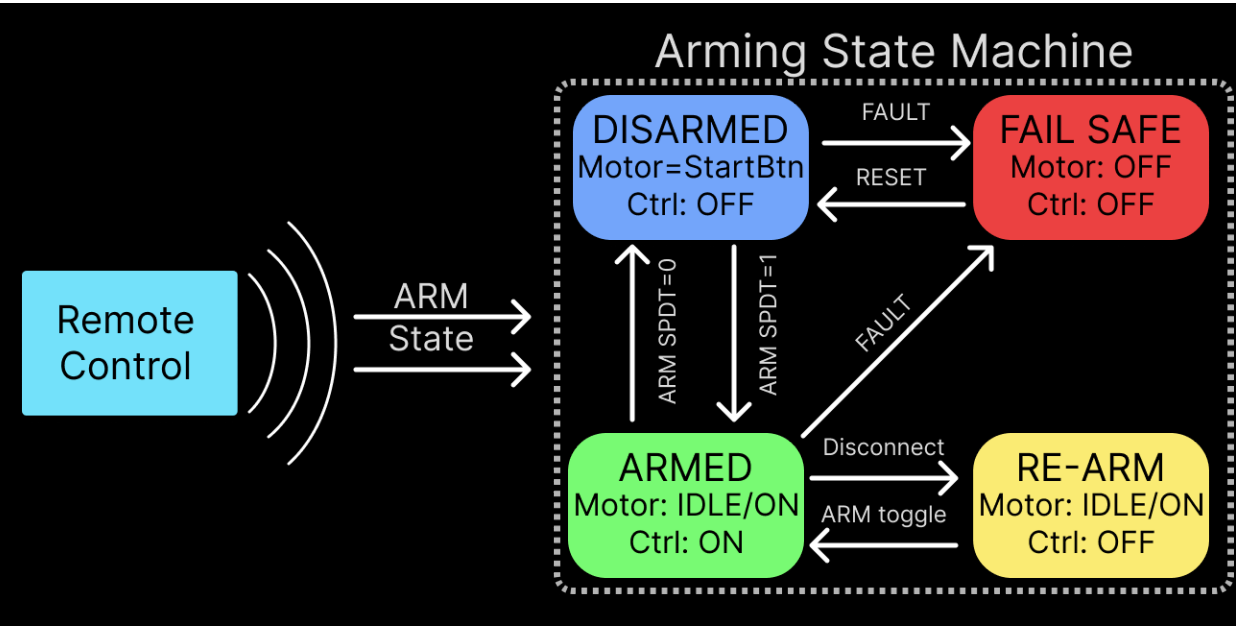


Figure 7: Flight Controller Receiver Firmware Block Diagram

3.2.4 Handheld Controller Firmware Block Diagram

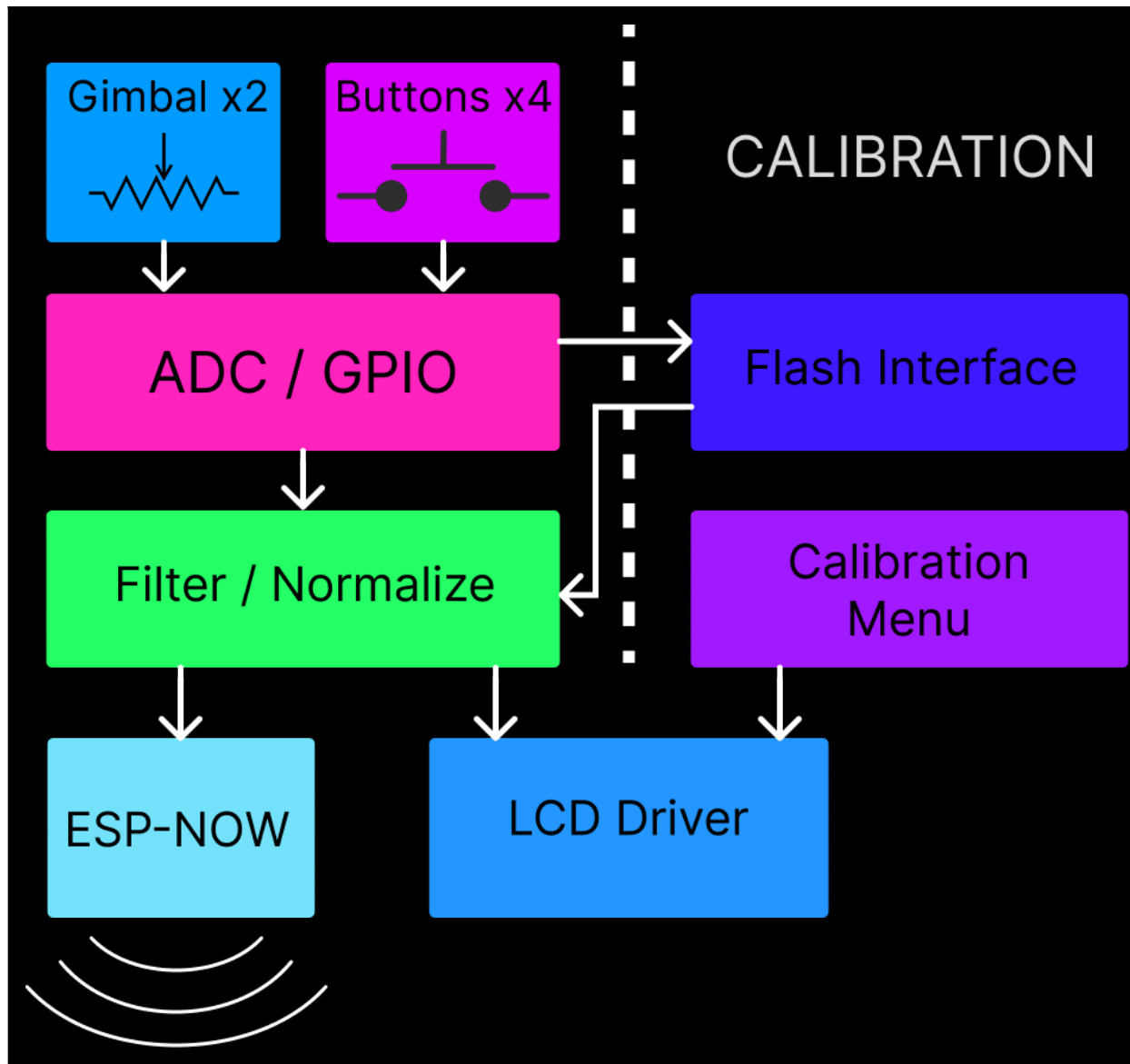


Figure 8: Handheld Controller Firmware Block Diagram

4. Block Descriptions

This section expands upon Section 3. Block Diagrams by describing the individual constituent parts of the diagrams.

4.1. Hardware Blocks

4.1.1. BLDC Electronic Speed Controller Hardware Blocks

The ESC/airframe board contains the power electronics, motor drive, sensorless commutation logic, and voltage regulation for the entire drone. The board-edge XT30 connector accepts the 2S LiPo battery (6.0–8.4 V operating range), and the raw battery voltage is distributed to the motor power stages and to a Texas Instruments TPS563209DDCR synchronous buck converter. The buck converter produces a regulated 3.3 V rail that powers the STM32F405 MCU, the DRV8329 gate driver logic, the BEMF sensing network, and the board-to-board 3.3 V output to the avionics board above. The XT30 connector is an off-the-shelf component and the TPS563209DDCR is an off-the-shelf IC, both integrated onto the team's custom board.

The central controller on this board is an STMicroelectronics STM32F405RGT6 ARM Cortex-M4 microcontroller [12] clocked from a 16 MHz external oscillator. It generates three-phase PWM signals for each motor using four independent advanced-control timers (TIM1, TIM8, TIM3, TIM4) in center-aligned mode at 20 kHz. It samples BEMF voltages via two on-chip ADCs (ADC1, ADC2) for zero-cross detection, receives throttle commands from the avionics board as four PWM input signals, monitors fault conditions via the DRV8329 nFAULT output, and controls the gate driver enable via the nSLEEP pin. The STM32F405RGT6 is an off-the-shelf IC integrated onto the team's custom board.

Each of the four motors is driven by a Texas Instruments DRV8329BREER three-phase smart gate driver [17] paired with a set of six CSD17581Q5A N-channel MOSFETs [18] (30 V, SON 5x6 package) arranged in a three-phase half-bridge topology — one high-side and one low-side MOSFET per phase. Each gate driver receives three PWM signals from the STM32 and drives the corresponding half-bridge gates. The DRV8329 provides hardware dead-time insertion via the DT pin, fault detection via the nFAULT output, overcurrent protection via the DRVOFF pin, and a configurable current sense amplifier. At any given commutation step, one phase is driven high, one is driven low, and the third is left floating for BEMF sensing. The MOSFET drain outputs connect directly to the motor phase terminals at the board corners. The gate drivers, MOSFETs, and current sense resistors are all off-the-shelf components integrated onto the team's custom board.

The back-EMF detection network was designed in-house. Each motor's three phase voltages are routed through resistor dividers ($R1 = 22 \text{ k}\Omega$, $R2 = 11 \text{ k}\Omega$) that scale the phase voltage into the STM32 ADC's 0–3.3 V input range, followed by second-order Sallen-Key low-pass filters that attenuate the 20 kHz PWM switching noise while passing the lower-frequency BEMF zero-cross signal. A virtual neutral point is reconstructed by averaging the three divided phase voltages. The zero-cross of the

floating phase relative to the virtual neutral indicates rotor position for commutation timing. A 100 Ohm series resistor at each ADC input provides transient protection.

Low-value sense resistors on the low-side return path of each half-bridge allow the DRV8329's internal current sense amplifier to monitor phase current. When the amplified sense voltage exceeds the overcurrent threshold, the DRVOFF pin is asserted to disable all gate outputs, protecting the MOSFETs and motor windings.

The board-to-board interface consists of soldered wire connections that carry the 3.3 V power rail and ground to the avionics board above, and four PWM motor command signals from the avionics board's ESP32-S3 to the STM32F405's PWM input capture channels.

4.1.2. Avionics/Flight Controller Hardware Blocks

The avionics board houses the flight controller, inertial sensors, and wireless communication module. The central controller is an Espressif ESP32-S3-WROOM-N16 module [14], which provides the processing for three concurrent firmware tasks: PID flight stabilization, ESP-NOW wireless communication (via the module's onboard 2.4 GHz PCB trace antenna), and failsafe management. The module outputs four PWM motor command signals to the ESC board below and runs the Espressif ESP-IDF framework. The ESP32-S3-WROOM-N16 is an off-the-shelf module integrated onto the team's custom board.

Two inertial sensors provide feedback for the control loop, both interfaced via SPI with separate chip select lines on a shared bus. The TDK InvenSense ICM-40609-D is a 6-axis MEMS inertial measurement unit [20] providing 3-axis gyroscope and 3-axis accelerometer data. The gyroscope provides angular rate measurements used directly by the PID inner rate loops, while the accelerometer provides gravity-referenced tilt data fused by the Mahony complementary filter for the outer angle loops. The MEMSIC MMC5983MA is a 3-axis magnetic sensor [21] that provides an absolute heading reference for the yaw axis, preventing gyroscope-only yaw drift over time. Both sensors are off-the-shelf ICs integrated onto the team's custom board.

A UART-based auto-programming circuit, designed in-house, allows firmware flashing without manual boot-mode button sequences. The board receives a regulated 3.3 V rail from the ESC board below through the board-to-board wiring, which powers the ESP32-S3 module, both inertial sensors, and all on-board logic. Additionally, the board includes redundant power circuitry capable of accepting the raw 7.4 V bus directly, which was used during early development when the avionics board was tested independently with a commercial off-the-shelf ESC.

The board-to-board interface consists of soldered wire connections to the ESC board below, carrying the 3.3 V supply and ground on the power side and four PWM output lines from the ESP32-S3 to the ESC board's STM32F405 on the signal side.

4.1.3. Handheld Controller Hardware Block Diagram

The handheld controller provides the pilot interface for manual flight control. The central controller is an Espressif ESP32-C3-Mini module [15], which reads joystick ADC inputs, drives the OLED display via I2C, reads button and switch GPIO inputs, and transmits ESP-NOW control packets to the drone at 100 Hz via the module's onboard 2.4 GHz PCB trace antenna. The antenna is oriented vertically when the controller is held in the normal operating position, maximizing radiation pattern coverage toward the drone. The ESP32-C3-Mini is an off-the-shelf module integrated onto the team's custom board.

Pilot input is provided by two potentiometer-based gimbals harvested from an IDEAFLY MC6 controller, providing four analog axes (throttle, yaw, pitch, roll). Each gimbal connects to the controller PCB via a JST connector and is read by the ESP32-C3's 12-bit ADC. The firmware applies IIR low-pass filtering, calibration, normalization to a signed 100-point range, and a deadband of three counts to eliminate stick drift near center. The gimbals are off-the-shelf components integrated into the team's custom controller.

A 0.96-inch monochrome OLED display (96 x 64 pixels), interfaced via I2C, provides real-time status feedback to the pilot. {STATUS: Confirm what the screen displays — e.g., connection status, armed/disarmed state, battery voltage, stick values.} Four panel-mounted push buttons and a single-throw toggle switch for arm/disarm are read as GPIO inputs by the ESP32-C3. The arm switch requires a physical action to enable motor output, serving as a safety interlock. The display, buttons, and switch are all off-the-shelf components.

The controller is powered by a single-cell LiPo battery with charging provided through a USB-C port connected to a dedicated charge management IC on the controller PCB. {STATUS: Charge IC part number, battery capacity in mAh.} A voltage regulation stage provides the regulated supply for the ESP32-C3 and peripherals. The charge circuit and voltage regulation were designed in-house using off-the-shelf ICs. The custom controller PCB was designed in Altium Designer and manufactured by JLCPCB,

integrating the ESP32-C3-Mini module and JST connectors for all peripheral connections onto a single board.

4.2. Firmware Blocks

The software for the project is largely separated into three distinct subsections: controller and communications, PID, and ESC architectures. Each, and their individual parts, are described in more detail following.

4.2.1. Remote Control Firmware Blocks

The firmware for the remote control can be defined by software making up the transmitter and receiver portions of the system. These, and their important blocks are expanded upon below.

4.2.1.1. Transmitter Firmware Blocks

The transmitter firmware runs on the handheld controller and is responsible for turning physical pilot inputs, including four analog joystick axes and buttons, into a stream of control packets transmitted to the drone over ESP-NOW. It is structured as a set of cooperating FreeRTOS tasks that sample, condition, packetize, and transmit pilot input at a fixed rate, with supporting subsystems for joystick calibration, persistent storage, on-screen feedback, and link-health monitoring. The blocks below describe each of these subsystems in turn, following the data-path from raw ADC sample to over-the-air packet.

4.2.1.1.1. System Initialization

When the system powers on, `app_main_sender()` brings up the hardware and software subsystems in a fixed order. NVS is initialized first, as it is used by both the Wi-Fi stack and by the joystick calibration loader. Wi-Fi is then started in station mode, and ESP-NOW is initialized afterwards [16]. The receiver ESP's MAC address is registered as a peer on a fixed channel with no encryption, keeping the latency low at the cost of security; this was deemed a reasonable tradeoff for short-range control links. The four joystick ADC channels are configured through the ESP-IDF one-shot ADC driver, and the user interface GPIOs are configured as inputs with internal pull-ups so they read high on idle. The I²C bus and SSD1306 OLED are initialized last, after which a greeting message is drawn to confirm that the display is alive. Finally, the system attempts to load a previously saved calibration from NVS. If none exist, a sane default based on the 12-bit ADC channel is loaded so that the controller is still usable on first boot.

4.2.1.1.2. Input Acquisition

The `input_task()` is the entry point for all user input to the system. It runs on a fixed 10ms period using `vTaskDelayUntil()` from FreeRTOS, giving a nominal 100Hz sample rate. This is limited by the default clock speed of the ESP32 chip being used. On each cycle, it samples all four joystick axes through the ADC. The arming switch is inverted on read so that the “up” idle position corresponds to disarmed. This is done to implement safety features in the event that the controller loses power. The task also implements a simple press-and-hold state machine on the calibration button: if the button is held for more than `CALI_HOLD_TIME`, two seconds in this case, while the controller is disarmed, it transitions the global `system_mode` to `MODE_CALIBRATE` and notifies the calibration task to take over. While in calibration mode, `input task()` idles so it does not combat the calibration routine for the ADC.

4.2.1.1.3. Signal Processing

Raw ADC samples tend to be noisy and uncentered, especially with the potentiometer-based gimbals being used in this project, so they are passed through three conditioning stages before becoming actual control values. First, a first-order IIR lowpass filter with an alpha value of 0.15 smooths high-frequency jitter from the ADC and from mechanical vibrations in the sticks. The small alpha value trades a bit of the responsiveness for a much cleaner signal. Second, `normalize_axis_calibrated()` maps the filtered raw value into a human-meaningful range using the stored calibration. Throttle is treated as a unipolar axis, mapped on 0 to 100, while yaw, pitch, and roll are bipolar and mapped on -100 to 100, using two separate linear segments of min-mid and mid-max. This compensates for joysticks whose mechanical center may not be exactly halfway between the endpoints of the ADC limits. Both branches clamp the result to guard against overshoot from noise or imperfect calibration. Finally, `apply_deadband()` zeros out any bipolar values within ± 3 of center, which eliminates the small twitching that would otherwise occur when the user is not touching the sticks. These conditioned and processed values are written into the shared `curr_inputs` structure type for the control task to consume.

4.2.1.1.4. Calibration Subsystem

Calibration exists because no two joysticks have the same electrical centre or travel range, and these values may also drift over time. The subsystem has three parts. The interactive routine `calibrate_joysticks()` walks the user through each axis in turn, and for each axis prompts them on the OLED to push the stick to the low position, then center, then high, capturing an ADC reading at each step when the calibration button is pressed. The routine includes a small debounce delay and waits for button release between captures so a single press cannot accidentally advance two steps. The wrapper task `joystick_calibration_task()` blocks on `ulTaskNotifyTake()` and only runs the

routine when `input_task()` notifies it, after which it persists the new calibration and returns the system to `MODE_CONTROL`. Persistence is handled by `save_cali_nvs()` and `load_cali_nvs()`, which store the entire `axis_calibration_t` struct as a single blob under the “joystick” namespace in NVS. `Set_default_calibration()` provides the fallback used on first boot or if NVS is corrupted.

4.2.1.1.5. Control and Packetization

The `control_task()` is the bridge between user intent and the ESP-NOW link. Also running at 100Hz, it copies the conditioned values from `curr_inputs` into a `drone_packet_t`, tags the packet as a control packet, `PKT_TYPE_CTRL`, and stamps it with a monotonically increasing sequence number and current microsecond-resolution timestamp from `esp_timer_get_time()`. The sequence number lets the receiver detect dropped or reordered packets, and the transmit timestamp lets it compute end-to-end latency once the clocks are synchronized. The task also enforces a critical safety rule: if the arming switch is not engaged, throttle is forced to zero before the packet leaves the controller, so a disarmed controller cannot command motor output even if downstream code misbehaves. The completed packet is then pushed into both `transmit_queue` and `oled_update_queue` using `xQueueOverwrite()`, which keeps only the most recent packet in the queue, which is exactly the desired behaviour for a real-time control system because stale packets have no value.

4.2.1.1.6. Communication and Transmission

The communication block has two tasks and one callback. `Transmit_task()` blocks on `transmit_queue` and forwards each control packet to the receiver over ESP-NOW. Using a blocking receive means the task consumes essentially no CPU when there is nothing to send. Running in parallel, `esp_now_sync_task()` emits a dedicated sync packet, `PKT_TYPE_SYNC`, every 500ms, carrying its own transmission timestamp so the receiver can maintain a running estimate of the clock offset between the two devices. The send callback, `esp_now_send_cb`, is registered with the ESP-NOW driver and is invoked by the Wi-Fi stack after every transmission attempt; it counts consecutive failures and raises a `link_warning` flag once the count crosses `PACKET_LOSS_WARNING_THRESHOLD`, giving the rest of the system a single boolean to react to link degradation without having to track radio events itself.

4.2.1.1.7. User Feedback and OLED

The `oled_update_task()` is the user-facing output during normal operation. It pulls the latest packet from `oled_update_queue` and, every half-second, logs the current stick values and the arming states. This half-second timing exists because the screen and ESP logging cannot keep up with the 100Hz control loop, and a human cannot read faster than that regardless. The task deliberately suspends the OLED writes whenever

system_mode is MODE_CALIBRATE, so the calibration routine has exclusive use of the display while it is walking the user through the axes.

4.2.1.1.8. System State Management

A small state machine in the global system_mode variable coordinates the tasks. MODE_CONTROL is the normal flight state in which input, control, transmit, and OLED tasks all run as described. MODE_CALIBRATE is entered by the earlier-mentioned long-press of the calibration button while disarmed, and causes input_task() and oled_update_task() to step aside so the calibration task owns the ADC and screen. Centralizing the mode in one variable keeps the inter-task coordination simple. Each task just checks the mode at the top of its loop rather than needing direct knowledge of what other tasks are doing.

4.2.1.1.9. RTOS Scheduling

Underpinning the above is the FreeRTOS scheduling layer. Tasks are assigned priorities that reflect their real-time importance: transmit_task() is highest at priority 4 because a delayed packet directly increases control latency; input_task() and control_task() sit at priority 3 since they feed the transmitter; the calibration and sync tasks run at priority 2; the OLED task is at priority 1; and an idle task at priority 0 ensures that the scheduler always has something to run. Communication between tasks uses two single-shot queues with overwrite semantics for the control stream to ensure only the freshest packet is held, task notifications for the lightweight handoff between input_task() and joystick_calibration_task(), and shared volatile structs (curr_inputs and system_mode) for states that are read far more often than they are written. This layering, with faster producers, prioritized consumers, and a clear separation between data path and control path, is what allows the controller to hit a consistent sub-10ms end-to-end response time.

4.2.1.2. Receiver Firmware Blocks

The receiver firmware runs on the drone's flight controller and forms the other half of the control link. Its job is to accept incoming ESP-NOW packets from the handheld transmitter, validate them, translate the pilot's stick deflections into physical setpoints (target angles and rates), and hand those setpoints to the PID loop in a form that is guaranteed to be safe to act on. Around this core data path sit several supporting blocks: a clock-synchronization mechanism for measuring true end-to-end latency, an arming state machine that gates motor output behind explicit operator action, a failsafe that disarms the aircraft if the link goes silent, and a link-quality monitor that tracks loss, latency, and jitter. The blocks below describe each of these in the order a packet encounters them, from Wi-Fi ISR to sanitized setpoint.

4.2.1.2.1. System Initialization

The receiver brings up its networking stack in two stages. `Comms_wifi_init()` initializes the network layer, the default event loop, and Wi-Fi in station mode with RAM-only storage, then locks the radio to a fixed channel that must match the sender.

`Comms_init()` is the entry point called by the flight controller's main. This stores the handle of the downstream `controller_queue` that is owned by the PID task, reads and logs the device MAC address for pairing, creates the single-shot ISR-to-task queue, registers the ESP-NOW receive callback, and spawns three worker tasks. It also logs `sizeof(drone_packet_t)` so that any struct-layout mismatch between sender and receiver firmware is immediately visible. A mismatched size causes every packet to fail the length check in the ISR and is otherwise very hard to diagnose.

4.2.1.2.2. ESP-NOW Receive Callback

This runs in the Wi-Fi driver's task context, which is considered to be of ISR priority, and is therefore kept deliberately short. It first rejects any frame whose length doesn't match `sizeof(drone_packet_t)`, which guards against stray broadcasts and version mismatches. It then branches on the packet type. Sync packets (`PKT_TYPE_SYNC`) are handled entirely in the callback: the receiver subtracts the sender's timestamp from its own current `esp_timer_get_time()` value to learn the clock offset between the two devices, stores it under the spinlock, and sets `has_offset` so downstream latency math becomes valid. Sync packets are not forwarded to the RX queue since they carry no control state and are effectively useless to the queue. Control packets are copied into the single-slot RX queue with `xQueueOverwriteFromISR()`, so the freshest packet always exists in the queue, and a slow consumer can never build up a backlog.

4.2.1.2.3. RX Task

The RX task is the first piece of "real" code that touches a packet, and has three jobs: latency checking, sequence checking, and statistics. For latency, it uses the clock offset learned from the sync packets to translate the sender's transmit timestamp into the receiver's clock domain and compute the true end-to-end delay; if that delay exceeds `MAX_ALLOWED_LATENCY_US`, the packet is considered stale and dropped. Before the first sync packet has been received, the offset is unknown and this gate is bypassed so the link can come up. For sequencing, it does a wrap-safe unsigned-16-bit subtraction against the previous sequence number: a change of zero is a duplicate (dropped), a change greater than 32768 is an out-of-order or rolled-back packet (dropped), and a change greater than one means packets were lost in flight, in which case the missing count is added to a running counter. For statistics, it updates min/max/sum/count of latency, plus a jitter estimator implemented as an EWMA over the absolute first difference of latency, giving a cheap running measure of link smoothness. Once a packet has passed all checks, it is copied into the shared

latest_rx_pkt, the last-receive timestamp is updated, and the has_rx flag is set so the controller task knows it has something fresh to act on. All shared-state writes happen under rx_mux, a portMUX_TYPE spinlock, because the same fields are touched from the Wi-Fi ISR context, and this is necessary to avoid race conditions.

4.2.1.2.4. Controller Task

The controller task converts validated wire packets into safe setpoints for the PID loop. It runs at 50Hz and begins each cycle by snapshotting latest_rx_pkt and the last-receive tick under the spinlock. The wire values are then scaled into physical units: throttle passes through unchanged as motor percent (0-100), pitch and roll are scaled from ± 100 to $\pm 30^\circ$ as angle-mode setpoints, and yaw is scaled from ± 100 to $\pm 200^\circ/\text{s}$ as a rate-mode setpoint. These limits are the hard envelope the pilot is allowed to command, and tightening them is the simplest way to make the drone tamer.

The arming state machine has four states. STATE_DISARMED is the power-on state; the drone enters STATE_ARMED only after observing a rising edge on the arm switch *while* the throttle is below 1% and at least MIN_GOOD_PACKETS consecutive packets have satisfied that condition. This prevents accidental arm-on-power-up if the switch happens to be in the armed position. STATE_ARMED is the only state in which stick inputs are passed through to the PID; a falling edge on the arming switch returns to STATE_DISARMED, and a heartbeat older than FAILSAFE_TIMEOUT forces the system into STATE_FAILSAFE. This state cannot be exited by simply restoring the link, but the operator must first toggle the arm switch off, which moves the system to STATE_WAIT_FOR_REARM. From there, a fresh rising edge with low throttle is required for rearm. This deliberately makes recovery a two-step manual action so a momentarily-lost-then-recovered link cannot spontaneously start the motors.

After the state update, the task constructs a safe_ctrl copy of the setpoints and zeros throttle, yaw, pitch, and roll whenever the state is anything other than STATE_ARMED. The sanitized struct is pushed into controller_queue with xQueueOverwrite(), again using overwrite semantics so the PID always sees the freshest setpoint and never an old queued one.

4.2.1.2.5. Stats Task

A low-priority background task that wakes once per second, snapshots the link-quality counters under the spinlock, and computes packet loss percentages and average latency. It is currently silent, as logging is commented out for the final stages of the project, but the snapshot itself is cheap and gives the rest of the system a single place from which to publish telemetry to a ground station, controller, or onboard log. Loss is

calculated as $loss = \frac{missed\ packets}{(received\ packets + missed\ packets)}$, rather than as a change of sequence numbers, which keeps the math correct across sequence wraps.

4.2.1.2.6. Shared State and Concurrency

Three execution contexts touch the same data: the Wi-Fi ISR via the receive callback, the RX task, and the controller and stats task. Coordination is handled by a single portMUX_TYPE spinlock, rx_mux, which protects latest_rx_pkt, last_rx_time, has_rx, the clock offset, and all link-quality counters. A spinlock rather than a mutex is used because one of the writers is an ISR and ISRs cannot block, and the system must handle the possible race conditions. Two single-slot queues handle the data path: rx_transmit_queue from ISR to RX task, and controller_queue from controller task to PID task. Both use overwrite semantics, which is the right choice for a real-time control stream; there is no value in delivering an old setpoint just because a newer one exists.

4.2.1.2.7. RTOS Scheduling

Three tasks run inside the comms layer with priorities chosen by latency sensitivity. Rx_task() runs at priority 4 because any delay here directly increases the freshness gap seen by the PID loop. Controller_task() runs at priority 3, just below RX, because it produces the setpoints the flight controller actually consumes. Stats_task() runs at a priority of 1, since dropped or delayed telemetry is harmless. Stack sizes are assigned to reflect each task's complexity, with the controller task getting the most because of its state machine and floating-point scaling.

4.2.2. PID Software Block

The PID block is the heart of the flight controller: it takes in the sanitized setpoints produced by the comms layer and the drone's measured attitude and rates from the IMU and magnetometer, and computes the per-motor outputs that drive the drone towards those setpoints. It is structured as a cascaded control loop: an outer angle loop for pitch and roll that produces rate targets, feeding an inner rate loop that runs on all three axes and produces torque commands. The throttle is mixed in at the final motor-mixing stage. Supporting logic handles integral windup, output saturation, and the motor-arming/idle state transitions requested by the pilot. The blocks below walk through each stage of this pipeline, from sensor input to motor command.

4.2.3. Initialization

Upon startup the system executes an initialization procedure. GPIO configuration and semaphore initialization are implemented. SPI configuration for the magnetometer and IMU are implemented. Magnetometer and IMU registers are configured to enable outputs, set device configuration and filtering, and characterize other device behaviour. The Mahony orientation filter is initialized to the output data rate of the IMU. The PID

structures are initialized to the tuned parameters. Motor initialization is commenced and limiting is applied to ensure uniform thrust across each motor. Protocols are initialized to enable communication with the controller. Tasks to enable magnetometer and IMU polling are initialized alongside the main control task

4.2.4. Main PID Task

PID implementation is implemented in a single main task on the flight controller ESP-32: `pid_task`. The task receives input data from the magnetometer, IMU, and controller through lightweight dedicated tasks polling interrupt pins and initiating SPI transactions pending data availability: `imu_task` and `imu_gpio_isr_handler`, and `magnetometer_task`. IMU and magnetometer axes are rebased in the control loop to ensure unification of the aircraft sensor axes and to account for the varied orientation of the physical sensors. Offsets and corrections are applied to the raw magnetometer and IMU data to remove sensor biases. The magnetometer output magnitude is continuously compared to the local magnetic field to schedule magnetometer integration into the Mahony filter to periods when magnetic noise from the motors or other sources is low and within a given band around the local field magnitude. The Mahony orientation filter is applied to the rebased, calibrated, and scaled data to derive the attitude of the drone with respect to the magnetic and gravitational references [22]. The cascaded PID architecture is applied and subject to the motor mixing algorithm before output as PWM signals to the motors. Outputs to the motors and updates to the PID are subject to conditioning to enforce controller states and prevent unwanted motor behaviour. The PID implementation features a feedforward term to increase responsivity, as well as additional modifications to prevent unwanted behaviours such as integral windup, persistence of the derivative term, and output limiting to prevent the motors from exceeding the operable range of the ESC.

4.2.5. ESC Software Blocks

4.2.5.1. Main Loop Bare Metal

The main loop starts every cycle by observing the `global_fault` to ensure proper operation and continue running. Following this motor speed control PWM signal is polled to calculate motor speeds and check if fault conditions are reported. The motor state machine is updated which includes performing startup when valid PWM is received. BEMF ADC buffered values are compared and utilized for possible commutation updates. The final main loop operation is to compute debug and stats values for tuning and validation.

4.2.5.2. Motor State Machine

IDLE → ALIGN → OPENLOOP_RUN → MOTOR_RUN → FAULT

IDLE: Motor inactive. Waiting for a `MOTOR_Start()`, which is based on receiving a valid PWM throttle signal (>4.8% duty).

ALIGN: Phase A is energized for 100ms to magnetically align the rotor to a known position before the open-loop ramp.

OPENLOOP_RUN: Fixed-timing open-loop commutation. The step delay ramps geometrically from 5000 μ s down to 240 μ s using a factor of 0.997 per step. Duty simultaneously ramps from 25% to 21%. BEMF sampling begins after a set wait period. Transition to MOTOR_RUN requires: openloop_hold_us to elapse (default 400ms), and BEMF synced (zc_sync_count = 12 consecutive valid zero crossings). If these conditions are not met timeout occurs after MOTOR_OPENLOOP_TIMEOUT_US = 4000ms, triggering Motor_Recovery().

MOTOR_RUN: BEMF-driven closed-loop commutation. Each zero crossing detected by the ADC ISR calls MOTOR_ZeroCrossDetected(), which advances the commutation step and updates commutation_delay_us from the measured ZC period. A fallback timer in the main loop commutates the motor if no BEMF event occurs within fallback time (margin widens to 50% for the first 200 BEMF commutations during the closed-loop transition).

FAULT: All motors disabled. Only an MCU reset recovers. The global_fault flag in the main loop immediately asserts all four nSLEEP pins low (killing all gate drivers) and enters a fast LED blink loop.

Recovery: Before permanently entering FAULT state, a failing motor attempts max_restarts (default 5) automatic restarts via Motor_Recovery(), which resets BEMF sync and re-enters ALIGN state. A restart during open-loop only faults that motor; a stall detected during MOTOR_RUN calls MOTOR_EmergencyShutdown() and kills all four motors.

4.2.5.3. Interrupt Structure

Fires at 52.5kHz triggered by TIM1_TRGO (update event), this fires all three ADCs simultaneously. Raw samples are written to `adc\_buf[motor][phase]`. BEMF processing is deferred to the main loop via the `adc\_new[]` flags to keep the ISR short.

PWM throttle inputs are driven by a shared ISR for all four PWM receive lines. Measure timestamp differences between rising and falling edges reporting valid frames if frames are received at least every 100ms. If invalid PWM read or timeout occurs, the motor is turned off (clears pwm_rx_valid). Duty is mapped to: below 960 μ s means 0% (motor stop); 960–1100 μ s means minimum duty 10%; 1100–2000 μ s means linear map 10%–98%; above 2000 μ s is capped at 98%.

4.2.5.4. Stall Detection

Stall is detected by counting fallback commutations. If the motor falls back to open loop commutation more than 300 times within a rolling 1000-commutation window, a stall is declared. Variables fault_motor and fault_cause are set for post-mortem debug inspection.

4.2.5.5. ADC/BEMF Multiplexing

All three ADCs are used in injected mode, each ADC has four injected ranks. Motor 4's three phases are distributed one per ADC on rank 4, so Motor 4 data is only marked valid once all three ADCs have fired. The other motors each have their own ADC that samples all three motor phases.

5. Engineering Analysis

This segment of the report describes engineering analysis tools used in the creation of the project.

5.1. Thrust and Weight Budget

5.1.1. Mass Budget

The fully assembled drone (with battery, all boards, motors, propellers, and 3D-printed parts) weighed in at **125 g**, well within the 250 g constraint (Requirement 4.1). A component-level mass breakdown is provided below.

Component	Quantity	Unit Mass (g)	Subtotal (g)
GAONENG Flight 2S LiPo (1000 mAh, 120C)	1	49	49
YSIDO 1104 8600 kV brushless motor	4	6.75	27
5-blade propellers (51 mm)	4	<1	4
Custom avionics board	1	15	15
Custom ESC board(s)	1	25	25
3D-printed parts (legs, guards)	1	5	5
Total			125

Table 1: Mass budget

Thrust Requirement. For stable manual hover, the minimum per-motor thrust must equal one-quarter of the total weight:

$$T_{hover} = \frac{W_{total}}{n_{motors}} = \frac{125g}{4\ motors} = 31.25 \frac{g}{motor}$$

Equation 1: Minimum hovering thrust calculation

For adequate control authority during maneuvers and disturbance rejection, a minimum thrust-to-weight ratio of 2:1 is targeted for quadrotor platforms.

Motor Capability. The YSIDO 1104 8600 kV motors produce approximately **75 g** of thrust per motor at maximum throttle on a 2S LiPo (7.4 V nominal) with 51 mm 5-blade propellers. The total maximum system thrust is:

$$T_{max} = 4\ motors \times 75 \frac{g}{motor} = 300g$$

Equation 2: Maximum Thrust Calculation

Thrust-to-Weight Ratio.

$$\frac{T}{W} = \frac{T_{max}}{W_{total}} = \frac{300g}{125g} = 2.4$$

Equation 3: Thrust-to-weight Ratio Calculation

A thrust-to-weight ratio of **2.4:1** exceeds the **2:1** minimum target, providing sufficient margin for stable hover (which requires approximately **42%** throttle) and responsive attitude corrections. At hover, each motor operates at approximately 31.25 g of thrust out of its 75 g maximum, leaving **58%** of available thrust as headroom for maneuvering and disturbance rejection.

5.2. PWM Motor Mapping

In the flight control board, the output of each of the motors must generate a PWM channel of 1000us to 2000us with a frequency of 50hz. The 50hz standard was chosen as the drone itself is a mechanical system, often reacting slowly to the outside disturbances. In order to do this, we needed at least 4 channels of ADC. The ESP32 S3 was perfect for the task, as the CPU was incredibly powerful, with many ADCs and

a dual-core processor. This also helps with the PID and communications, as it has built in communication.

5.3. PID Tuning - Ziegler-Nichols INspired Heuristic

PID tuning was completed under inspiration from the method first researched by Ziegler and Nichols in 1942 [23]. Compared to standard PID, our control system was modified to implement feedforward control term, allowing fraction of the rate controller input to bypass the control loop, a modification common to systems that require a rapid response [24]. Tuning involved first tuning the inner loop of the cascaded PID architecture, starting with the proportional terms. The goal was to push the controller to generate a rapid response to changes in the gyro rates while avoiding oscillatory behaviour. Next proportional tuning was implemented for the outer PID loop, once more pushing the system to a rapid response without oscillatory behaviour. From this point live testing commenced with the addition of a small derivative term on the inner rate loops, starting at one fortieth of the tuned values for the proportional terms on the inner loop. The outer loops were not tuned for derivative control. After tuning the proportional and derivative terms, we proceeded to tune the feedforward terms until the stick movements were responsive to inputs without overwhelming the stabilization provided by the PID corrections. At this point, integral tuning could be implemented if desired, however, in our specific case integral tuning was not implemented as the steady state error of the system was deemed acceptable in operation. One note is that the yaw loop does not subscribe to the cascaded architecture; conventionally it is convenient for your drone to maintain the orientation that the pilot leaves it in, as opposed to having the drone direction locked. The final PID parameters are showcased in the following table. It is important to note that these are simply the values that we have arrived upon and are not necessarily perfect. These parameters are sufficient to enable hovering and movement but may be improved.

	K_p	K_i	K_d	K_f
Pitch Angle	6.0	0.0	0.0	0.0
Pitch Rate	4.0	0.0	0.05	0.5
Roll Angle	6.0	0.0	0.0	0.0
Roll Rate	2.0	0.0	0.05	0.5
Yaw Rate	0.3	0.0	0.015	0.2

Table 2: Final PID Tuning Parameters

5.4. Space Considerations

Our hard cutoff for the physical space of the drone itself was 10x10 cm, with a maximum weight of 250g. The purpose behind this is a mix of cost and practicality. Our problem statement dictates the size constraint of our drone, namely the nimbleness and maneuverability that a small drone would provide. The specific 10x10 cm cutoff with wings was due to cost constraints of JLCPCB. The cost goes up dramatically on any PCBs over this limit, and therefore, it was cost-effective and mission critical that the drone fit in this specification. The 250g limit is specified by the regulations of Transport Canada, which is detailed in Section 8: Regulatory Codes. The consideration of space was both a decision in mission criticality, cost-effectiveness and regulatory awareness.

6. Enclosure

The F-17 Humanitarian Hummingbird uses 3D-printed components for structural support and safety, supplementing the PCB-as-frame airframe architecture. All 3D models were designed using SolidWorks and Fusion 360, and printed on a team member's personal 3D printer using PLA Pro and TPU filaments.

6.1 Overall Structural Stackup

The airframe is organised into two distinct stackups: a motor–propguard–PCB–leg stack at each of the four corners, and a flight controller–ESC PCB stack at the centre.

At each motor corner, the 3D-printed structural components and the motor are clamped together by nylon screws into a single unit. From bottom to top the order is landing leg, ESC/airframe PCB, propeller guard, and motor, with the screws passing through every layer. Nylon fasteners were chosen over metal to save weight and to eliminate any risk of shorting exposed traces on the PCB. Because the motor sits directly on top of the propeller guard, its compliant patterned base is interposed between the motor and the PCB, adding a further stage of vibration damping that attenuates motor-induced oscillations before they reach the airframe board.

The flight controller and ESC PCB, by contrast, are soft-mounted to maximise vibration isolation for the onboard sensors. The two boards are joined structurally by four rubber dampeners seated on top of nylon standoffs, so at no point are they rigidly coupled, and all signal connections between them are routed through braided silicone wires along the sides rather than through rigid headers. This arrangement has produced roughly twice as stable IMU readings as the previous hard-mounted iteration.

6.2 Drone Landing Legs

Four identical landing legs, 3D-printed in thermoplastic polyurethane (TPU), provide ground clearance for the propellers and absorb impact forces during landings and incidental crashes. Each leg sits at the bottom of the motor corner stack, forming the lowest layer through which the nylon assembly screws pass, so it can be replaced independently without disturbing the rest of the structure. TPU was chosen over rigid filaments like PLA or PETG, which tend to fracture under sudden compressive loads: as an elastomer with a low flexural modulus and elongation at break in excess of 500 percent, it undergoes substantial elastic deformation on touchdown and then recovers its original shape, effectively acting as a small nonlinear spring in series with the airframe rather than transmitting the full impact into the carbon fiber frame, PCB stack, and motor bells.

6.3 Propeller Guards

Four PLA Pro propeller guards are mounted at the motor corners, each sandwiched between the ESC/airframe PCB and the motor and secured by the same nylon screws that clamp the stack together. Each ring shields the propeller arc from walls and obstacles and is supported by four curved struts, whose curvature distributes bending stress more evenly than straight spokes. The circular mounting base is a patterned ring of thin webs rather than a solid disc, acting as a compliant spring that attenuates high-frequency motor vibration before it reaches the flight controller. The four rings are tied together by H-shaped joints that protect the propeller in case of flip over as well as raise torsional rigidity and spread localised impacts across all corners, while a central spring feature where the joints converge redirects asymmetric impact loads across the diagonal of the airframe. Weak points at the strut roots, H-joint fillets, and screw bosses were progressively reinforced across iterations in response to simulation and test-flight results.

6.4 Handheld Controller Enclosure

The handheld controller enclosure is a two-piece 3D-printed shell in PLA Pro that houses the controller PCB, dual-stick gimbals, 1S LiPo battery, and OLED display. The controller PCB mounts to screw-in standoffs inside the enclosure. The shell includes cutouts for the four panel-mounted push buttons, the arm/disarm toggle switch, the USB-C charging port, and the display window. The enclosure was designed to be ergonomic for two-handed grip during flight operation.

6.5 Design Iterations

All 3D-printed parts went through one to two print iterations. Revisions primarily addressed fit tolerances between the printed parts and the PCB, clearance for components on the board surface, and screw hole alignment in the motor corner stack. No parts required more than two iterations to reach their final geometry.

7. Printed Circuit Boards

This section presents the three custom PCBs developed for the project. The team produced 7 custom PCBs designed during the project. Each board is described in its own subsection below.

7.1 Custom BLDC ESC / Airframe PCB

The ESC PCB was designed in Altium Designer and manufactured by JLCPCB. This board serves a dual role as both the electronic speed controller and the primary structural airframe of the drone (see Section 4.1). It carries the full ESC power electronics - three-phase half-bridge power stages for four motors, **DRV8329BREER** gate drivers, **CSD17581Q5A** MOSFETs, shunt based current sensing, BEMF detection network with **Sallen-Key filters**, the **STM32F405RGT6** microcontroller, and TPS563209DDCR buck converter - as well as the XT30 battery connector and motor mounting pads at each corner.

The board measures 87 mm × 87 mm and uses a 6-layer stack-up. The layout is clearly partitioned into two regions: a central signal region containing the MCU, gate drivers, and BEMF sensing, and a power region routed along the outer edges near each motor mount where the phase currents are delivered.

In the signal region, the six layers are arranged as signal-ground-power-signal-ground-signal, sandwiching every routing layer against an adjacent reference plane to preserve signal integrity through the densely packed traces around the MCU and gate drivers. In the power region, the same six layers are instead stitched together between phase, power, and ground copper, effectively paralleling all layers to achieve the copper weight and trace width required to carry the full phase currents from the MOSFETs out to the motor pads without excessive resistive loss or heating.

The design went through three iterations. The first two iterations were single-motor test boards used to validate the ESC power stage, gate driver configuration, and BEMF detection circuit on an individual motor before committing to the full four-motor design. An error in MOSFET orientation on the first test board required a respin (see Appendix VI for schedule impact). Each test board order cost approximately \$10 for five boards from JLCPCB. The third and final iteration is the full four-motor airframe board. The final board order cost approximately \$550 for 15 boards, six of which were assembled with JLCPCB's PCBA service. {STATUS: Lead time - approximately 2 weeks from order to delivery.}

7.2 Custom Avionics / Flight Controller PCB

The avionics PCB was designed in Altium Designer and manufactured by JLCPCB. This board stacks on top of the ESC/airframe board and houses the ESP32-S3-WROOM-N16 flight controller, the ICM-40609-D IMU, the MMC5983MA magnetometer, power regulation circuitry, and the auto-programming circuit for firmware flashing (see Section 4.2).

The board measures 46 mm x 85.7 mm and uses a 4-layer stack-up, with inner ground and power planes providing noise isolation between the sensitive inertial sensors and the digital logic.

The design went through two iterations. The first iteration was a development board that included a different IMU, the ICM 20648 intended for wearables applications, and extended wing sections that allowed motor mounting directly on the avionics board for preliminary PID development and testing before the ESC/airframe board was ready. The second and final iteration removed the wing sections - since the board now stacks on the ESC/airframe board which carries the motors - and incorporated the final sensor suite (ICM-40609-D IMU and MMC5983MA magnetometer). {STATUS: Cost and lead time — fill in from invoice.}

7.3 Custom Handheld Controller PCB

The handheld controller PCB was designed in Altium Designer by Minh Do, with the schematic developed jointly by Minh Do and Ethan Sam, and manufactured by JLCPCB. The board integrates the ESP32-C3-Mini wireless module, JST connectors for all peripherals (dual-stick gimbals, four panel-mounted buttons, arm/disarm toggle switch, 0.96-inch OLED display), a USB-C charging circuit with dedicated charge management IC, and a 1S LiPo voltage regulation stage.

The board measures {STATUS: mm × mm} and uses a 4-layer stack-up with inner ground and power planes for noise isolation and clean ADC routing from the joystick gimbals. The design was completed in a single iteration with no respins required. {STATUS: Cost and lead time — fill in from invoice.}

8. Regulatory Codes

The following regulatory codes and standards would apply if the F-17 Humanitarian Hummingbird were developed beyond the prototype stage and brought to market in Canada.

Transport Canada's Canadian Aviation Regulations (CARs) Part IX [1] governs the registration, certification, and operation of RPAS (drones) under 25 kg in Canadian airspace. Under current regulations, drones weighing less than 250 g flown recreationally in uncontrolled airspace are exempt from drone registration and pilot certification requirements. However, operators must still comply with altitude limits (122 m / 400 ft AGL), no-fly zones near airports and aerodromes, and visual line-of-sight requirements [1]. For non-recreational operations, even sub-250 g platforms may require registration and an RPAS pilot certificate depending on the class of operation. A commercial product would need to include operating guidance consistent with these rules.

Transport Canada TP 15263 [2] defines the knowledge standards for the Small Basic and Small Advanced RPAS pilot certificates. While not required for recreational operation of the sub-250 g F-17 prototype, any future commercial or advanced operation (e.g., beyond visual line-of-sight, over bystanders, or in controlled airspace) would require the operator to hold the appropriate certificate [2].

ISED RSS-247, Issue 2 [3] establishes the technical requirements for licence-exempt digital transmission systems operating in the 2.4 GHz ISM band, including maximum transmit power limits (1 W EIRP for the 2.4 GHz band), spurious emission requirements, and equipment certification procedures [3]. The ESP-NOW wireless link operates at 2.4 GHz and falls under this standard. The Espressif ESP32-S3-WROOM-N16 and ESP32-C3-Mini modules carry pre-existing ISED certifications, simplifying compliance provided the modules are used within their certified operating conditions and no modifications are made to the antenna systems.

ISED RSS-Gen, Issue 5 [4] establishes the general administrative and technical requirements applicable to all radio apparatus certified in Canada, including labelling requirements, user manual content, and RF exposure (SAR) limits [4]. Any radio-equipped product sold in Canada must comply with RSS-Gen in addition to the applicable equipment-specific standard.

ISED ICES-003 [5] sets conducted and radiated emission limits for digital apparatus sold in Canada [5]. The custom PCBs in the F-17 contain high-speed digital circuitry (MCU clocks, SPI buses, 20 kHz PWM switching) that generates unintentional electromagnetic emissions. A commercial product would require EMC testing to demonstrate that emissions remain within the Class B limits. The equivalent U.S. standard is FCC Part 15 Subpart B [6]; both are typically tested concurrently.

FCC Part 15 [6] would apply if the platform were sold in the United States, it would need to comply with FCC Part 15 Subpart B (unintentional radiators — applicable to the digital circuitry) and Subpart C (intentional radiators — applicable to the 2.4 GHz ESP-NOW transmitter) [6]. The Espressif modules carry FCC certification, which satisfies Subpart C when the modules are used as certified.

IEC 62368-1 [7] is the current hazard-based product safety standard for ICT and multimedia equipment, replacing the older IEC 60950-1 [7]. It applies to the drone's electronic assemblies and would govern electrical safety, energy-related hazards, and thermal safety in a commercial product. The Canadian adoption is CSA C22.2 No. 62368-1.

EU Directive 2011/65/EU, commonly known as RoHS [8], restricts the use of lead, mercury, cadmium, hexavalent chromium, polybrominated biphenyls (PBB), and polybrominated diphenyl ethers (PBDE) in electronic products sold in the European Union [8]. Compliance is increasingly expected globally. A production-intent design should specify RoHS-compliant components throughout the bill of materials.

UN 38.3 [9] specifies eight safety tests (altitude simulation, thermal cycling, vibration, shock, external short circuit, impact/crush, overcharge, forced discharge) that lithium-ion and lithium-polymer cells must pass for lawful transport [9]. The GAONENG Flight 2S LiPo battery used in the F-17 must be UN 38.3 certified by the cell manufacturer, and a commercial product would require appropriate shipping documentation and packaging compliant with IATA Dangerous Goods Regulations.

9. Design Alternatives Considered

This section details alternative options that the team discussed before finalizing the project's design.

9.1. Control Strategy: Cascaded PID vs. LQR vs. MPC

A cascaded PID control architecture was selected for attitude stabilization over two more advanced alternatives: linear-quadratic regulator (LQR) and model predictive control (MPC).

LQR is a full-state feedback controller that computes optimal gain matrices by minimizing a quadratic cost function over the system's state and control inputs. It requires an accurate linear state-space model of the quadrotor dynamics, including moment of inertia estimates for all three axes, motor thrust constants, and drag coefficients. While LQR can deliver superior disturbance rejection and coordinated multi-axis response compared to independently tuned PID loops, it demands a level of plant modeling that the team did not have time to develop. Characterizing the moment of inertia of a 145 g PCB-based airframe with asymmetric mass distribution (battery on one side, stacked boards in the center) would have required either CAD-based inertia estimation with accurate material properties or experimental measurement using a bifilar pendulum or similar test fixture. Any error in the plant model directly degrades LQR performance, and the model would have needed to be re-derived after each hardware revision. The team judged that the modeling effort was not justified given the project timeline and the fact that a well-tuned cascaded PID is sufficient for stable manual hover.

MPC is a receding-horizon optimization strategy that solves a constrained optimal control problem at each time step, predicting the system's future trajectory over a finite horizon and selecting the control input that minimizes a cost function subject to state and actuator constraints. MPC offers advantages for trajectory tracking, obstacle avoidance, and operating near actuator limits. However, it is computationally expensive and is poorly suited to the ESP32-S3's processing capabilities at a 1 kHz loop rate. Commercial drone platforms that implement MPC typically run it on dedicated companion computers or high-performance flight controllers with floating-point DSP units, and use MPC only for the outer position loop at 50 to 100 Hz while retaining PID or rate controllers for the fast inner loops. For a manual-hover application with no trajectory planning or position control, MPC offers no practical benefit over PID while introducing significant implementation complexity and computational risk.

The cascaded PID architecture was selected because it is well-understood, straightforward to tune using the Ziegler-Nichols method on the physical hardware, computationally lightweight (approximately 20 microseconds per control cycle for all six loops), and does not require an analytical plant model. The dual-loop structure (outer angle, inner rate) provides the key practical benefit of disturbance rejection on the fast inner loop while maintaining stable attitude tracking on the outer loop. This architecture is used by the majority of open-source and commercial flight controllers, including Betaflight [25] and PX4, confirming its suitability for quadrotor stabilization.

9.1. ESC: Custom Design vs. Commercial Off-The-Shelf

Early in the project, the team evaluated whether to use commercial off-the-shelf ESC modules or to design a custom ESC from scratch. A custom design was selected to meet the educational objectives of ENEL 400, to gain hands-on experience with power electronics and motor control firmware, and to maintain full control over the commutation strategy and interface with the avionics board. The resulting custom ESC, based on the STM32F405RGT6 and CSD17581Q5A MOSFETs with the DRV8329 gate driver, was designed, fabricated, and successfully integrated into the flying platform. The custom ESC ultimately outperformed the commercial off-the-shelf ESC that had been used as a temporary stand-in during early flight controller development. The trade-off was a significantly higher development effort and a test board respin that impacted the schedule (see Appendix VI), but the custom design provided the team with deeper understanding of three-phase motor drive, sensorless commutation, and real-time embedded systems.

9.2. ESC MCU: STM32 vs. ESP32

The STM32F405RGT6 was selected as the ESC microcontroller over an ESP32-based alternative. The primary drivers were the STM32's advanced timer peripherals [11], which natively support center-aligned complementary PWM - critical for safe and efficient three-phase motor drive. The STM32F405 also offers fast 12-bit ADCs with configurable trigger sources, enabling cycle-by-cycle BEMF sampling synchronized to the PWM period. By contrast, the ESP32's timer and ADC peripherals are primarily designed for general-purpose IoT applications and lack the deterministic, low-latency interrupt handling required for real-time motor commutation. Additionally, STMicroelectronics provides a mature 6-step sensorless motor control reference library (STSW-SPIN3204) [10] that served as a starting point for the firmware architecture, reducing development risk.

9.3. Commutation Strategy: BEMF Sensorless vs. Hall-Sensored

Sensorless BEMF zero-cross detection was chosen over hall-effect sensor commutation. The sensorless approach eliminates the need for three hall-effect sensors per motor, reducing wiring complexity (three fewer signal wires per motor), component cost, and potential failure points. Standard commercial drone motors, including the COTS YSIDO motors used in this project, do not include hall sensors, so a sensorless strategy was required for compatibility. The primary trade-off is that sensorless commutation requires an open-loop startup sequence (as the BEMF signal is not present at zero speed), and low-speed performance is limited. For a drone application where motors operate predominantly at medium to high speeds during flight, this trade-off is acceptable.

9.4. Flight Controller: Custom vs. Commercial (Betaflight/INAV)

A custom flight controller was developed rather than using a commercial off-the-shelf platform running Betaflight or INAV firmware [25]. The decision was driven by the educational objectives of ENEL 400, which require original design work including a custom PCB and embedded system development. A custom controller also allowed tight integration with the team's ESP-NOW communication stack and provided a flexible platform for experimentation with control algorithms and sensor configurations. The trade-off is significant: Betaflight and INAV represent years of community-driven optimization with robust PID tuning tools, blackbox logging, and proven flight performance. The custom controller required substantially more development time and produced a less mature control stack, but it fulfilled the course's design and learning objectives.

9.5. Wireless Protocol: ESP-NOW vs. CRSF/SBUS/Custom RF

ESP-NOW was selected as the wireless communication protocol between the handheld controller and the drone. Conventional RC protocols such as Crossfire (CRSF), Serial Bus (SBUS), and Pulsed Position Modulation (PPM) require a dedicated radio receiver module on the drone and a compatible transmitter, adding weight, cost, and additional wiring. ESP-NOW is native to the ESP32 microcontroller already present on the avionics board [16], meaning zero additional hardware was required for the drone side. The protocol provides low-latency peer-to-peer communication with sufficient throughput for real-time stick data at the required update rates. A fully custom RF physical layer was considered but rejected due to the significantly higher development effort with no clear advantage for a 10 m indoor line-of-sight requirement. The primary trade-off is that ESP-NOW is less proven for

drone control than established RC protocols and lacks built-in features such as RSSI (Received Signal Strength Indicator) telemetry and channel hopping.

9.6. Airframe: PCB-as-Frame vs. Traditional Frame

The team chose to use the PCB itself as the primary structural element of the drone, rather than mounting electronics onto a traditional carbon fiber or injection-molded plastic frame. The PCB-as-frame approach offers significant advantages for a microdrone: it eliminates the separate frame and the associated wiring harness, reducing total weight and assembly complexity. Power distribution is handled directly through copper traces, and motor mounting pads are integrated into the board layout. The approach also maximizes integration density, keeping the overall footprint within the 10 cm x 10 cm requirement. The trade-offs include reduced impact tolerance compared to a carbon fiber frame, limited ability to service or replace individual components without desoldering, and geometric constraints on arm length and motor placement imposed by the rectangular PCB form factor.

10. Future Work

The following work would be required to advance the F-17 Humanitarian Hummingbird from its current prototype state toward a complete and robust application platform.

10.1. UWB Indoor Positioning Reintegration

The ultra-wideband (UWB) ranging subsystem using the Qorvo DWM3000 was descope during this project to prioritize core flight capability. Future work should reintroduce the UWB tag and multi-anchor system, validate three-dimensional positioning accuracy, and fuse UWB position estimates with IMU data for autonomous station-keeping and indoor localization.

10.2. Payload Module

The original requirements specified modularity and the ability to support add-on modules (3.1.1.5). A standardized mechanical and electrical expansion interface should be designed, and a representative payload module (e.g., a camera or sensor breakout) should be demonstrated.

10.3. Regulatory Compliance Path

For commercialization, formal EMC pre-compliance measurements against ISED RSS-247 and FCC Part 15 should be conducted, and a certification path (CSA, CE, or equivalent) should be pursued.

10.4. Manufacturing and Cost Optimization

The current prototype uses multiple separate PCBs (drone airframe, avionics, ESC, controller). A production-intent revision should consolidate subsystems onto fewer boards, optimize the bill of materials for volume pricing, and address design-for-manufacturing considerations.

10.5. Autonomous Flight

The current platform provides manual attitude-mode flight only. Future work toward autonomous operation would involve adding altitude sensing (barometer or time-of-flight rangefinder) for altitude hold, integrating optical flow or UWB for horizontal position hold, and eventually upgrading the state estimator from the Mahony filter to an extended Kalman filter to fuse additional sensor modalities. The existing cascaded PID architecture and modular board stack are designed to accommodate these additions with minimal firmware changes - the outer loops would simply receive setpoints from an autonomous position controller rather than from the pilot's stick inputs.

11. References

[1] Transport Canada, "Canadian Aviation Regulations (SOR/96-433), Part IX — Remotely Piloted Aircraft Systems," Government of Canada. [Online]. Available: <https://laws-lois.justice.gc.ca/eng/regulations/SOR-96-433/page-67.html>

[2] Transport Canada, *TP 15263 — Knowledge Requirements for Pilots of Remotely Piloted Aircraft Systems*, Ottawa, ON, Canada, 2019.

[3] Innovation, Science and Economic Development Canada, *RSS-247, Issue 2: Digital Transmission Systems (DTS), Frequency Hopping Systems (FHS) and Licence-Exempt Local Area Network (LE-LAN) Devices*, Ottawa, ON, Canada, 2017.

- [4] Innovation, Science and Economic Development Canada, *RSS-Gen, Issue 5: General Requirements for Compliance of Radio Apparatus*, Ottawa, ON, Canada, 2018.
- [5] Innovation, Science and Economic Development Canada, *ICES-003, Issue 7: Information Technology Equipment (ITE) — Limits and Methods of Measurement*, Ottawa, ON, Canada, 2020.
- [6] Federal Communications Commission, "Title 47 CFR Part 15 — Radio Frequency Devices," U.S. Government Publishing Office, Washington, DC, USA.
- [7] *Audio/Video, Information and Communication Technology Equipment — Part 1: Safety Requirements*, IEC Standard 62368-1, 3rd ed., 2023.
- [8] European Parliament and Council, "Directive 2011/65/EU on the restriction of the use of certain hazardous substances in electrical and electronic equipment (RoHS)," *Official Journal of the European Union*, L 174, pp. 88–110, Jul. 2011.
- [9] United Nations, *Recommendations on the Transport of Dangerous Goods — Manual of Tests and Criteria*, Section 38.3: Lithium Metal and Lithium Ion Batteries, 7th rev. ed., Geneva, Switzerland, 2019.
- [10] STMicroelectronics, "STSW-SPIN3204: 6-Step Firmware Example for 3-Phase BLDC Motor Driving Based on the STSPIN32F0," Application Note, Geneva, Switzerland, 2020. [Online]. Available: <https://www.st.com/en/embedded-software/stsw-spin3204.html>
- [11] STMicroelectronics, *RM0090: STM32F405/415, STM32F407/417, STM32F427/437 and STM32F429/439 Advanced Arm-Based 32-Bit MCUs Reference Manual*, Rev. 19, Geneva, Switzerland, 2021.
- [12] STMicroelectronics, *STM32F405xx / STM32F407xx Datasheet*, DS8626, Rev. 9, Geneva, Switzerland, 2020.
- [13] Espressif Systems, *ESP-IDF Programming Guide*, v5.1, Shanghai, China, 2024. [Online]. Available: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/>
- [14] Espressif Systems, *ESP32-S3-WROOM-1 / ESP32-S3-WROOM-1U Datasheet*, Rev. 1.4, Shanghai, China, 2024.
- [15] Espressif Systems, *ESP32-C3-MINI-1 Datasheet*, Rev. 1.2, Shanghai, China, 2023.

- [16] Espressif Systems, "ESP-NOW User Guide," Shanghai, China, 2023. [Online]. Available:
https://www.espressif.com/sites/default/files/documentation/esp-now_user_guide_en.pdf
- [17] Texas Instruments, *DRV8329 50-V, 3-Phase Smart Gate Driver Datasheet*, SLVSGM2, Dallas, TX, USA, 2023.
- [18] Texas Instruments, *CSD17581Q5A 30-V N-Channel NexFET Power MOSFET Datasheet*, SLPS575, Dallas, TX, USA, 2018.
- [19] Texas Instruments, *TPS563209 3-A Synchronous Step-Down Converter in SOT-23 Package Datasheet*, SLUSE94, Dallas, TX, USA, 2020.
- [20] TDK InvenSense, *ICM-40609-D High-Performance 6-Axis MEMS MotionTracking Device Datasheet*, DS-000314, Rev. 1.6, San Jose, CA, USA, 2022.
- [21] MEMSIC, *MMC5983MA ± 8 Gauss, High Performance, Low Cost 3-Axis Magnetic Sensor Datasheet*, Rev. C, San Jose, CA, USA, 2019.
- [22] R. Mahony, T. Hamel, and J.-M. Pflimlin, "Nonlinear complementary filters on the special orthogonal group," *IEEE Transactions on Automatic Control*, vol. 53, no. 5, pp. 1203–1218, Jun. 2008, doi: 10.1109/TAC.2008.923738.
- [23] J. G. Ziegler and N. B. Nichols, "Optimum settings for automatic controllers," *Transactions of the ASME*, vol. 64, no. 11, pp. 759–768, Nov. 1942.
- [24] K. J. Åström and T. Hägglund, *PID Controllers: Theory, Design, and Tuning*, 2nd ed. Research Triangle Park, NC, USA: Instrument Society of America, 1995.
- [25] Betaflight Open Source Flight Controller Firmware. [Online]. Available:
<https://github.com/betaflight/betaflight>

12. Appendix I: Schematic Diagrams

This appendix holds the schematic diagrams for the PCBs created for this project.

12.1. Electronic Speed Controller Board Schematic

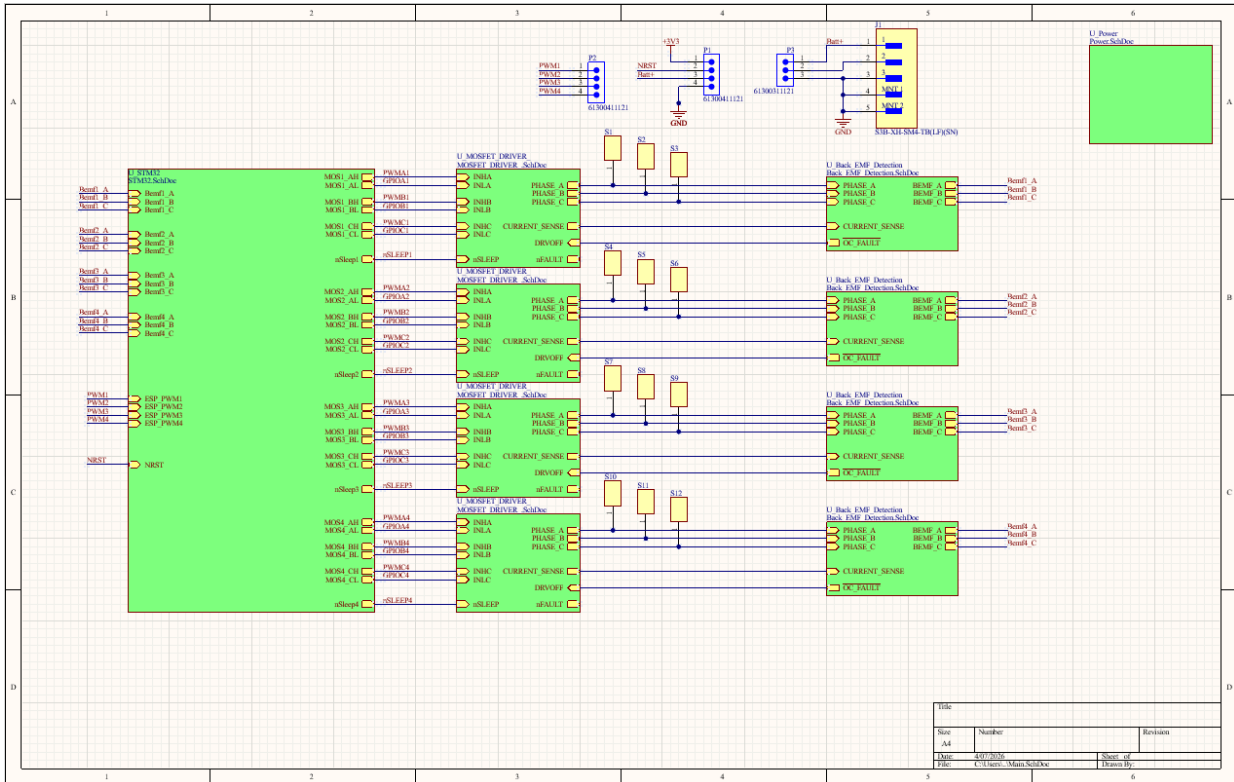


Figure : ESC Main Schematic

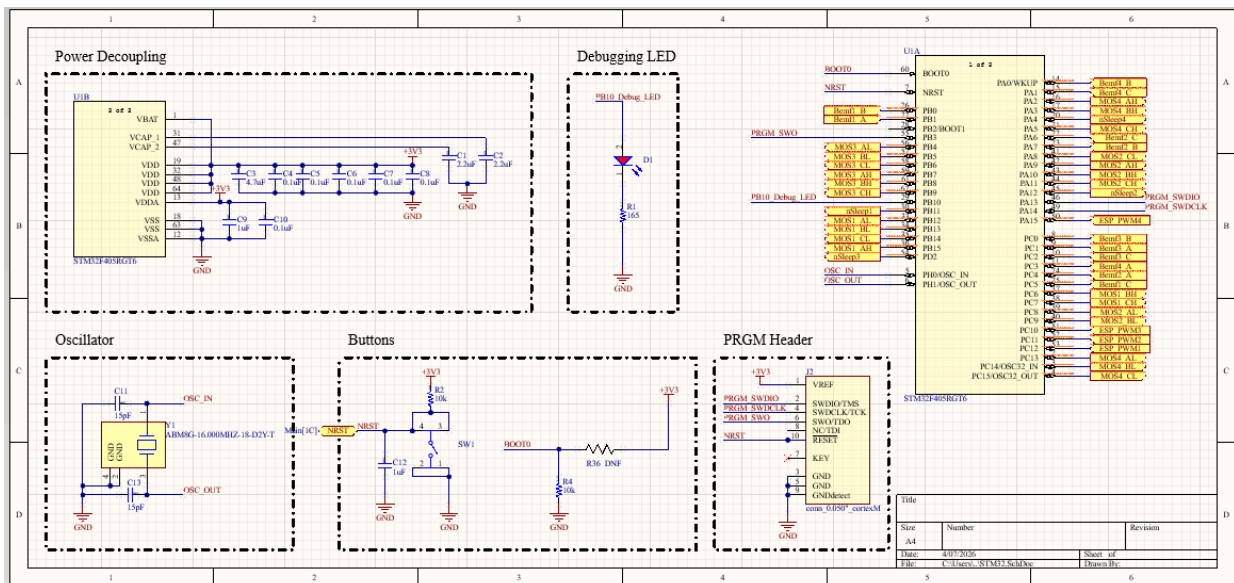


Figure : ESC MCU Schematic

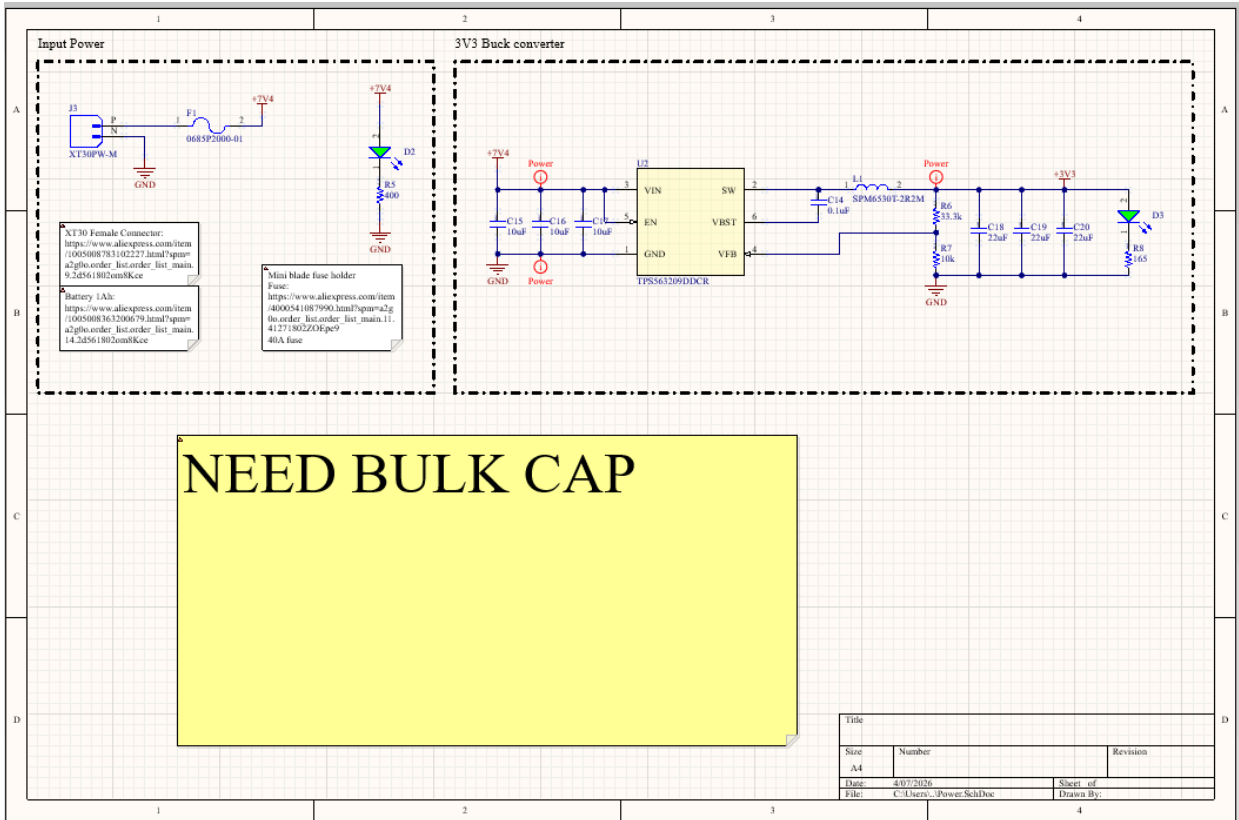


Figure : ESC Power Schematic

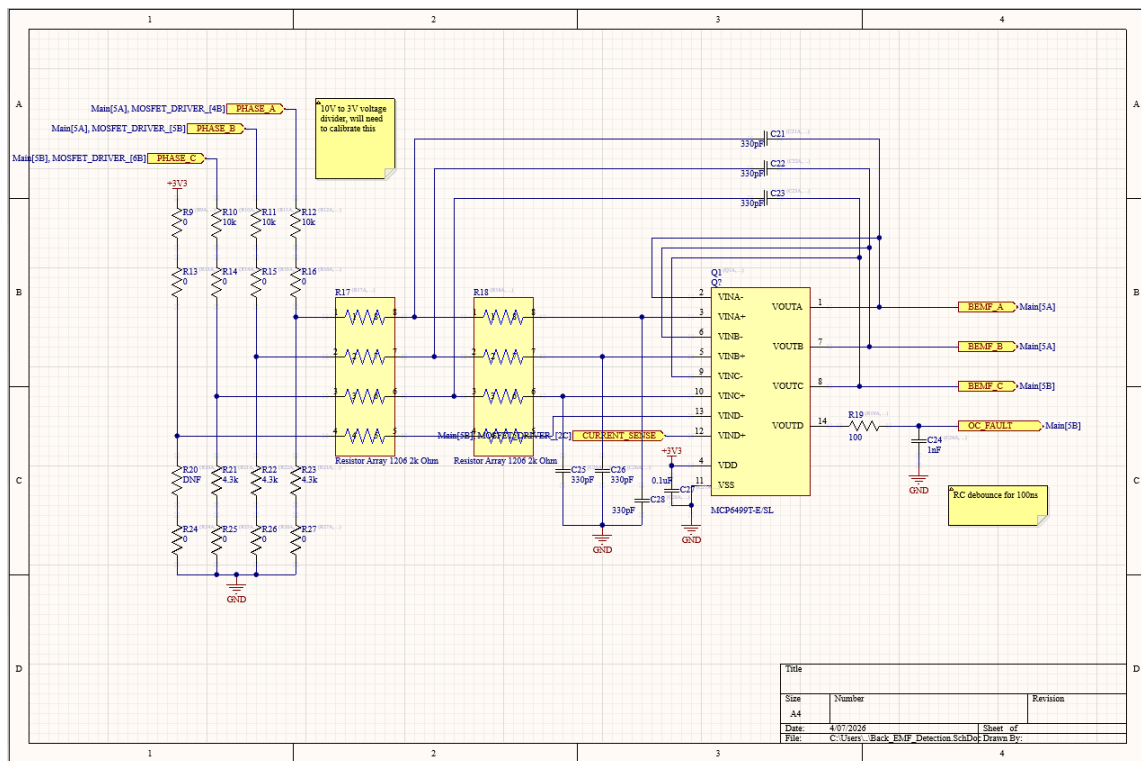


Figure : ESC Back EMF Schematic

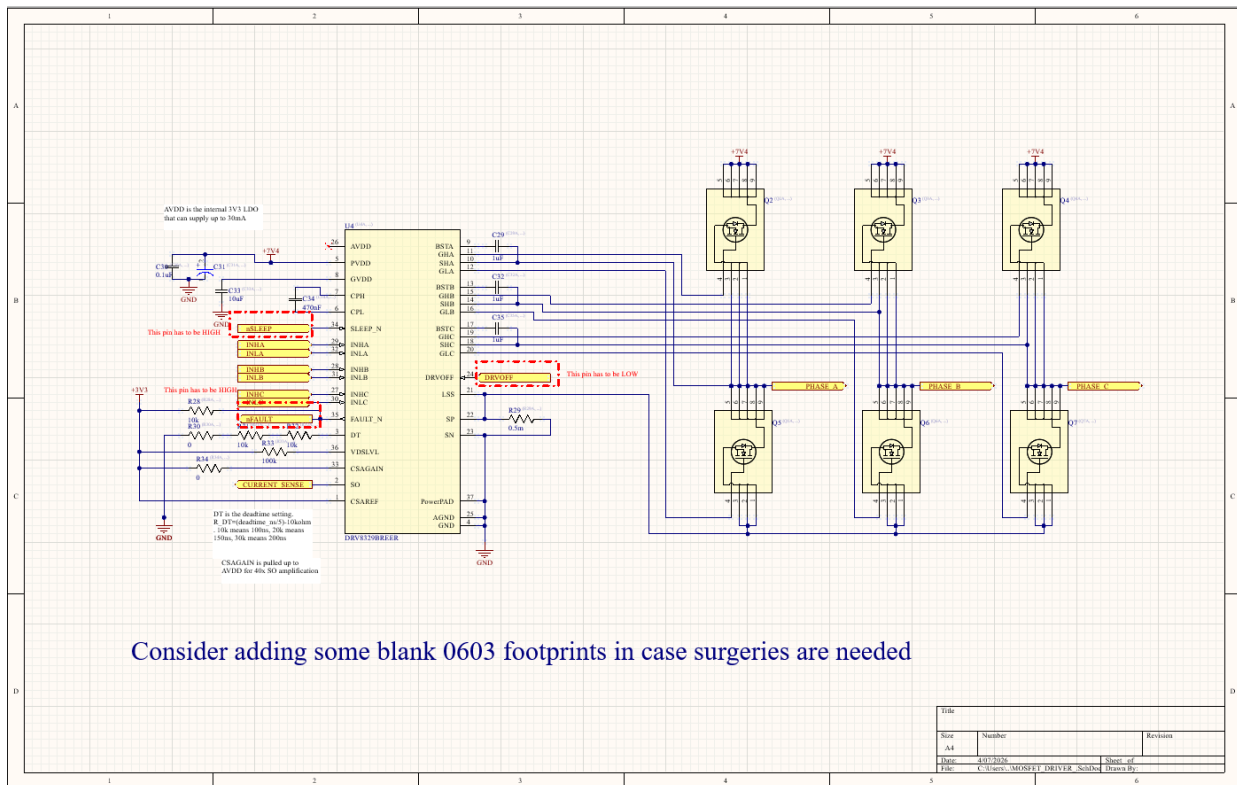


Figure : ESC MOSFET Driver Schematic

12.2. Flight Controller Board Schematic

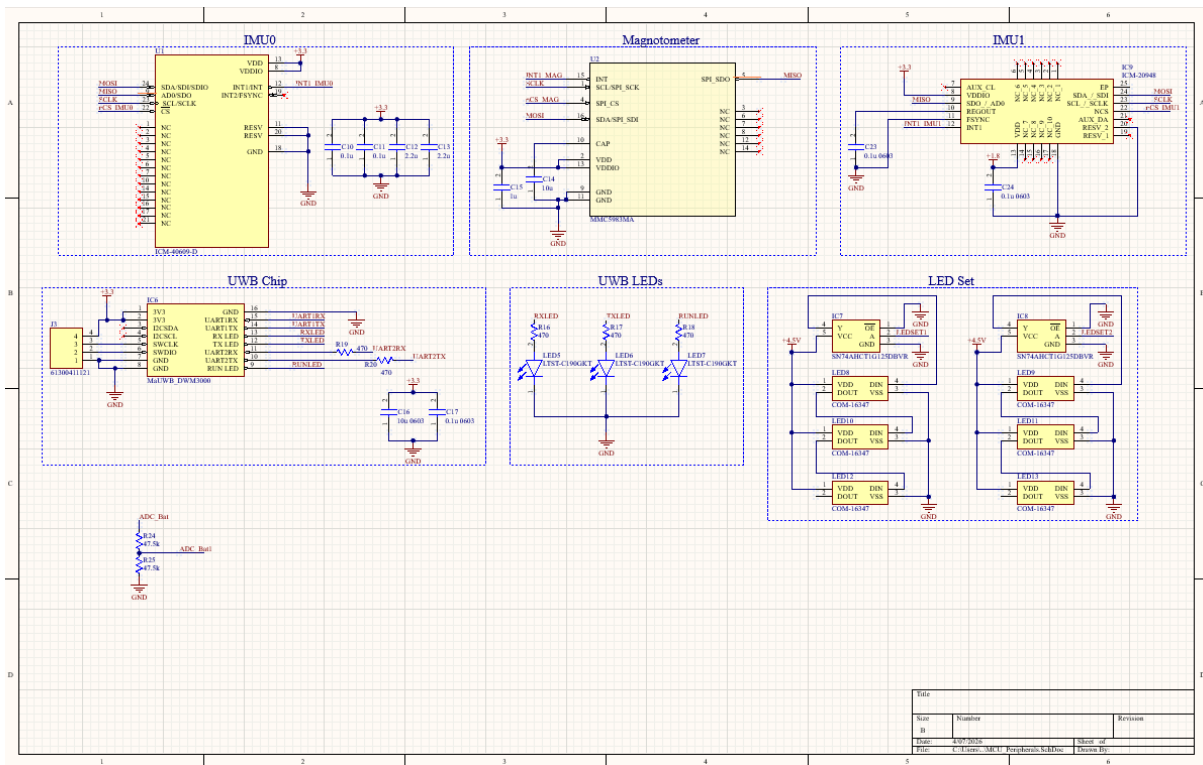
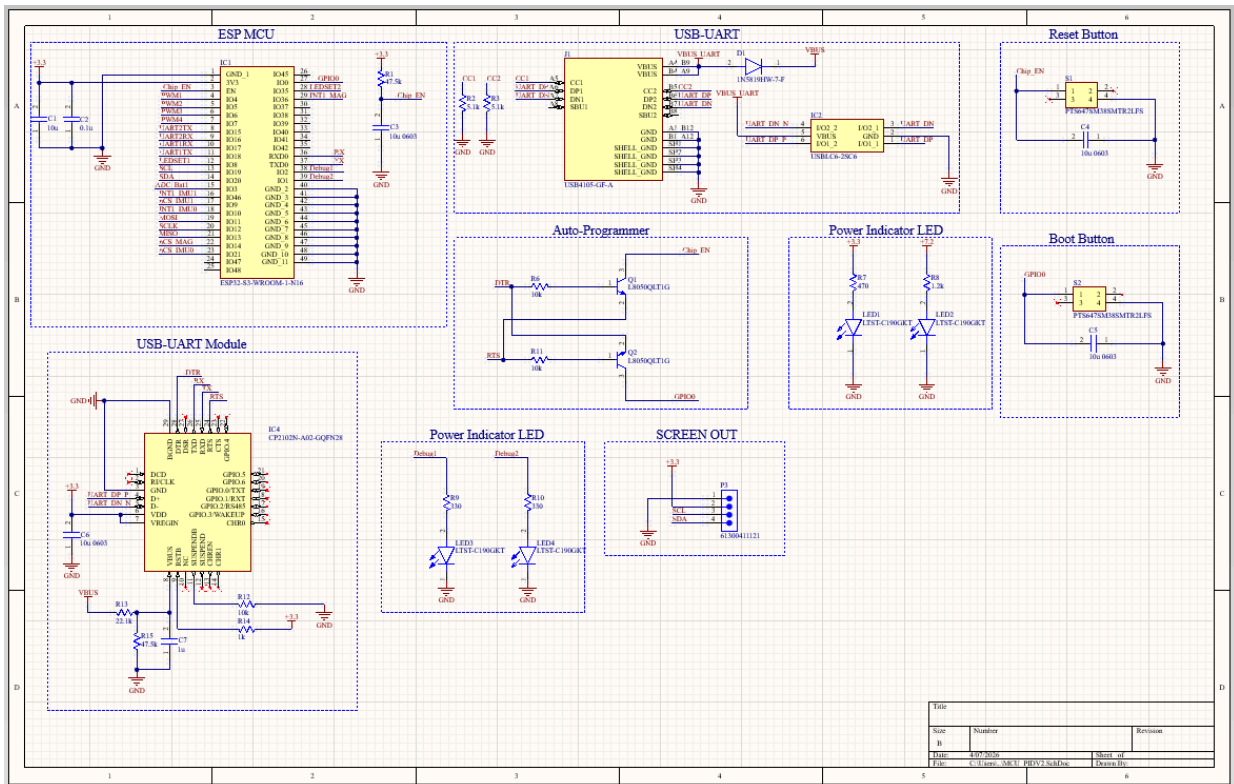


Figure : Flight Controller Peripherals Schematic

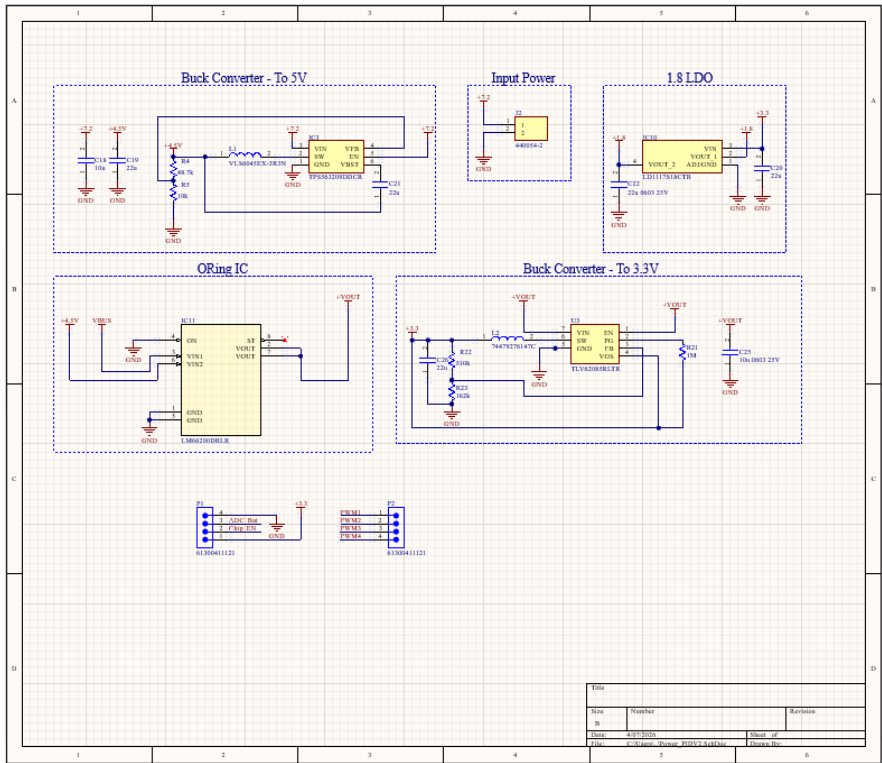


Figure : Flight Controller Power Schematic

12.3. Remote Control Board Schematic

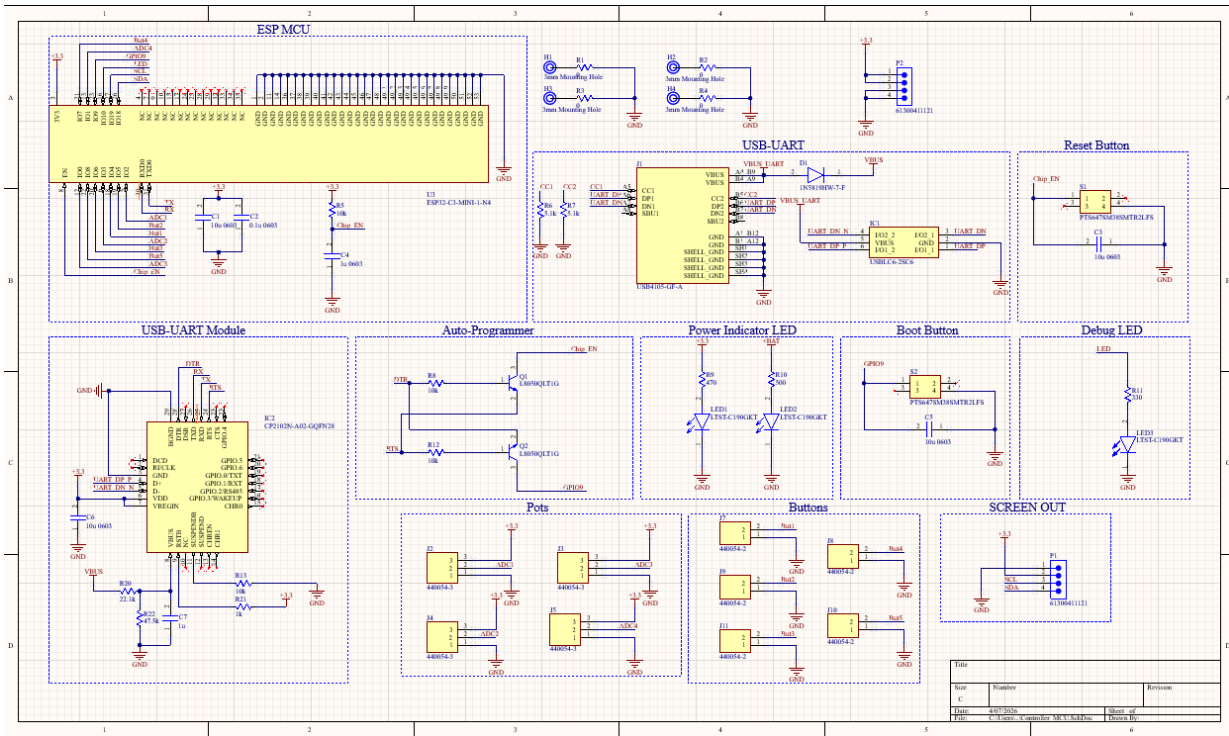


Figure : Controller MCU Schematic

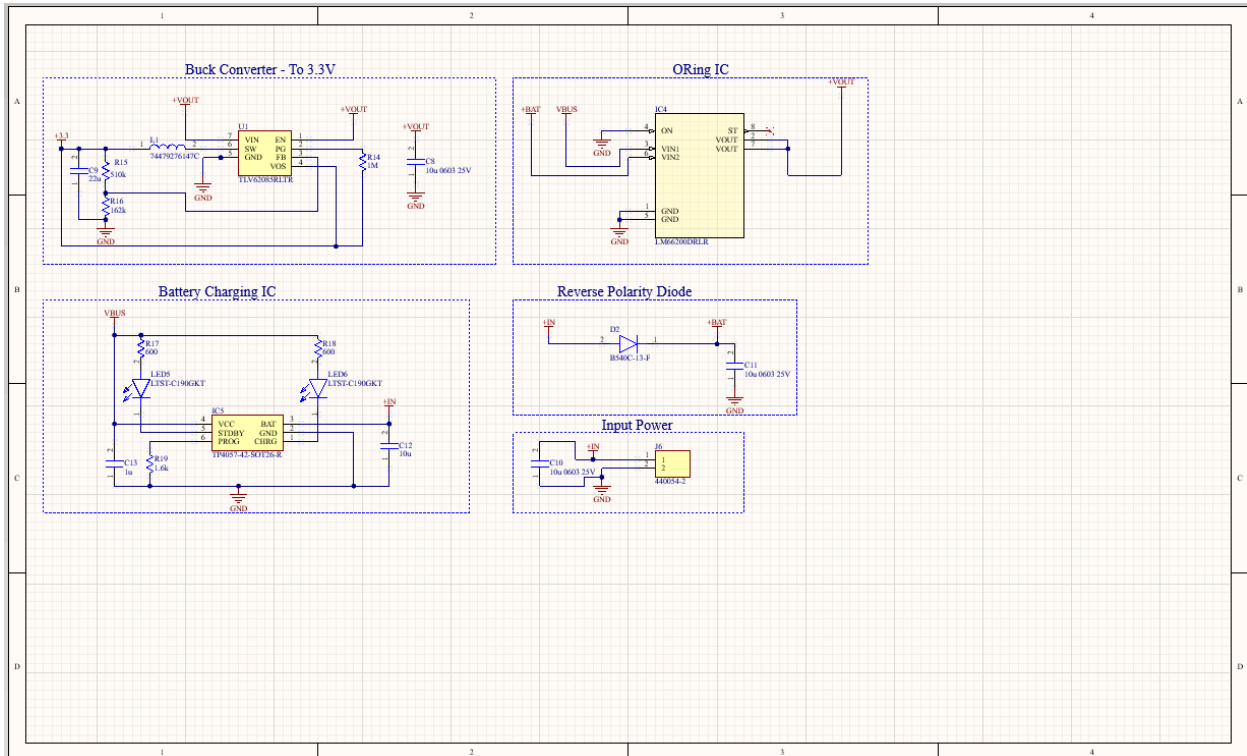


Figure : Controller Power Schematic

13. Appendix II: PCB Layouts

Enclosed in this appendix are the layouts for the PCBs used in the project.

13.1. Electronic Speed Controller Board Layout

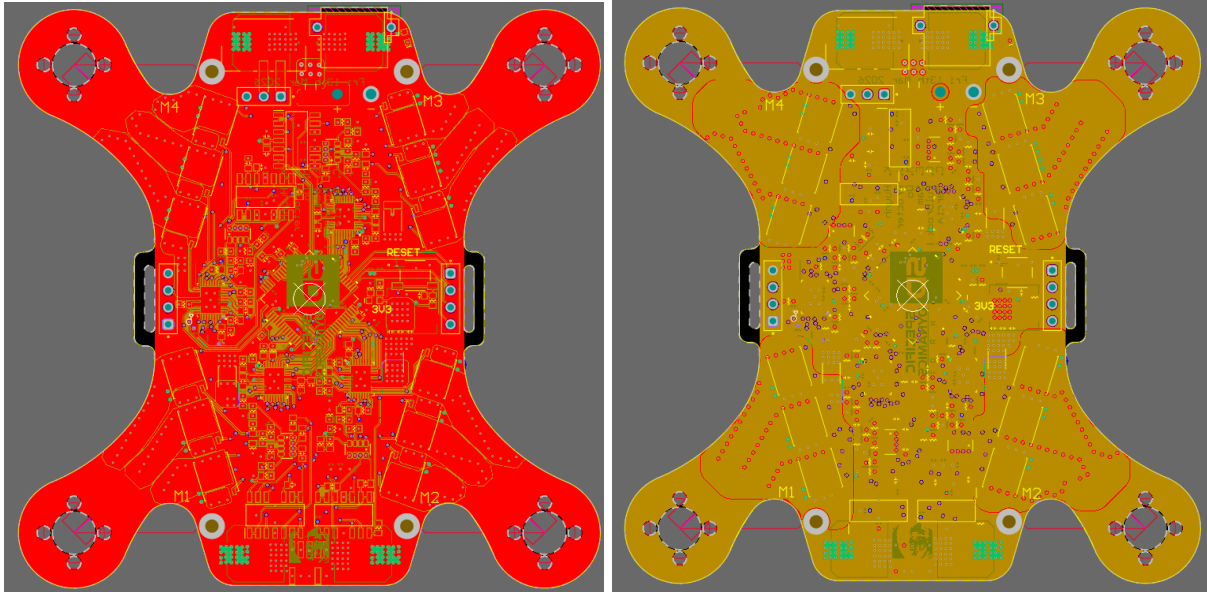


Figure : ESC layer 1 (SIGNAL1) and layer 2 (GND)

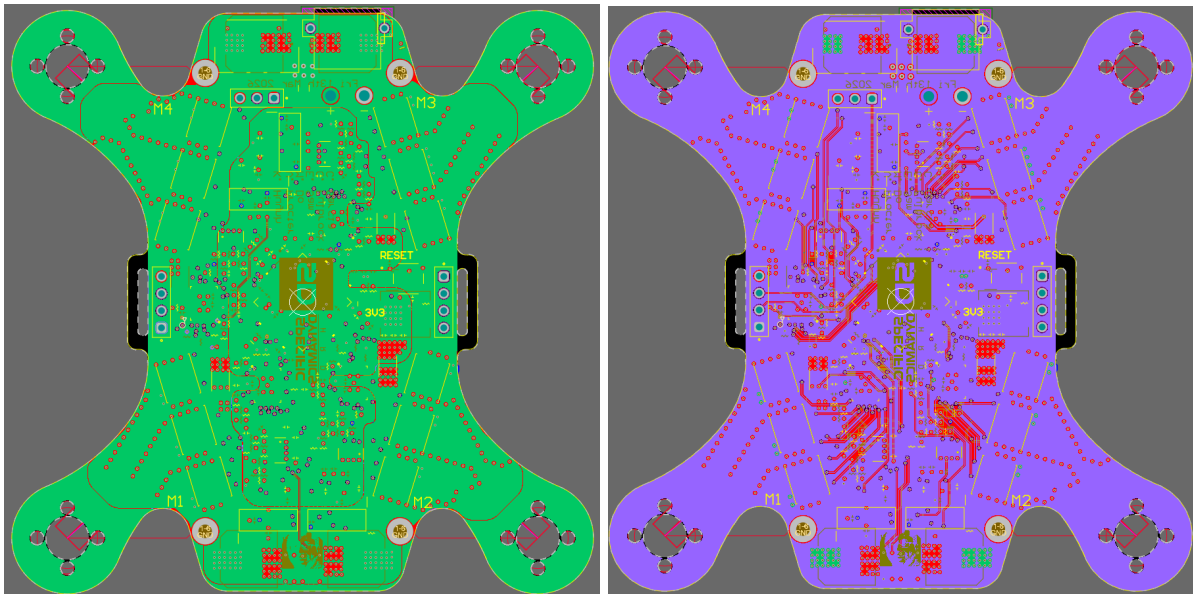


Figure : ESC layer 3 (POWER) and layer 4 (SIGNAL2)

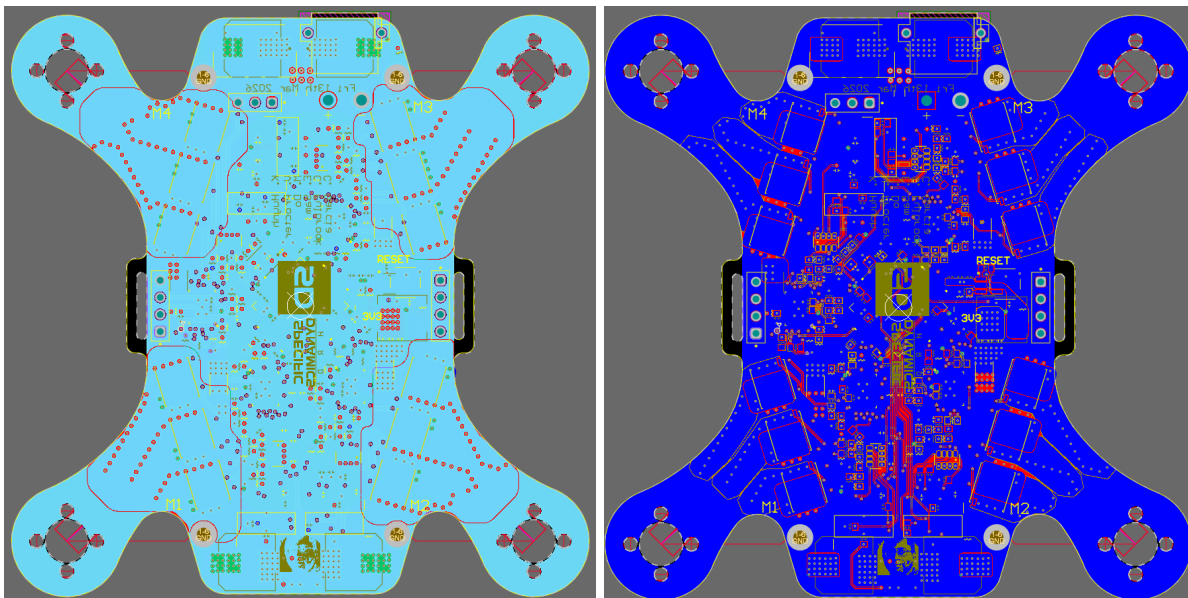


Figure : ESC layer 5 (GND) and layer 6 (SIGNAL3)

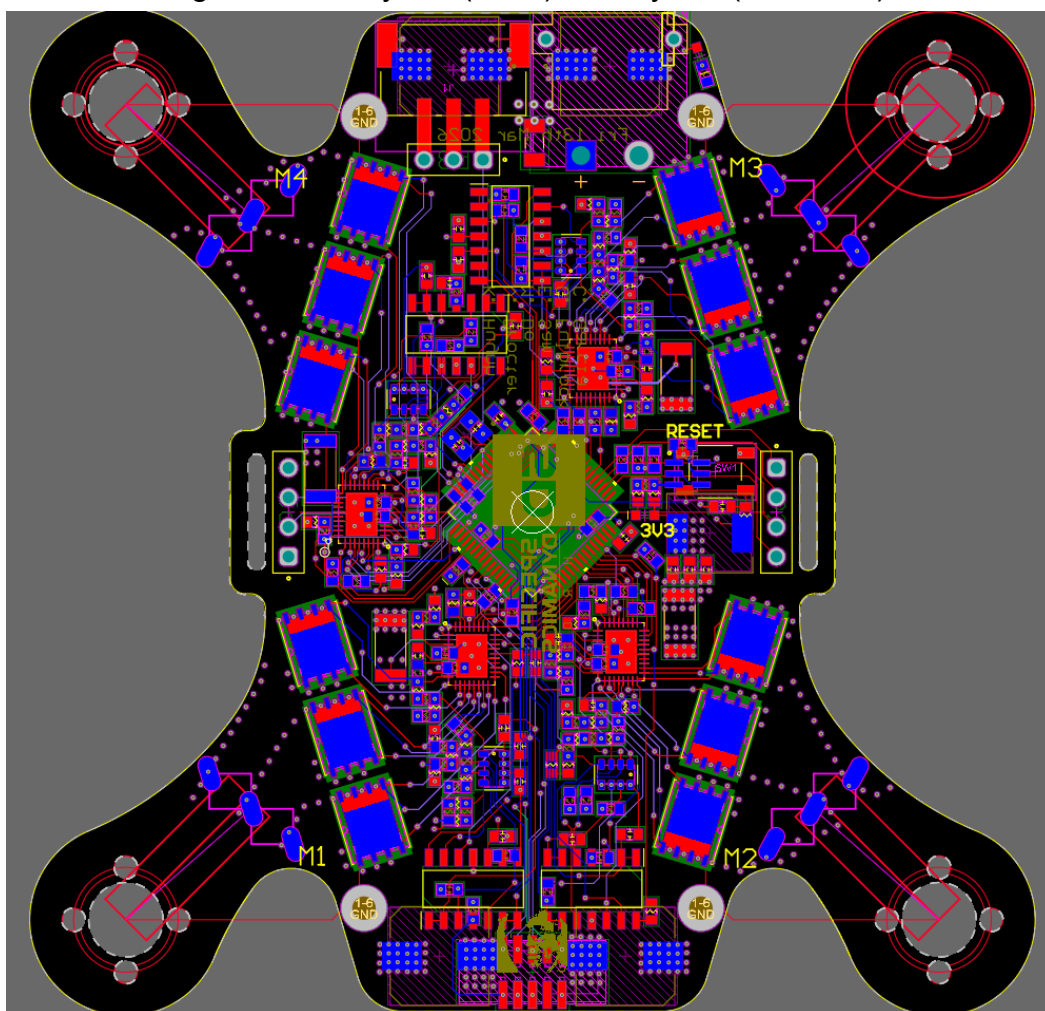


Figure : Signal view with all polygon hidden

13.2. Flight Controller Board Layout

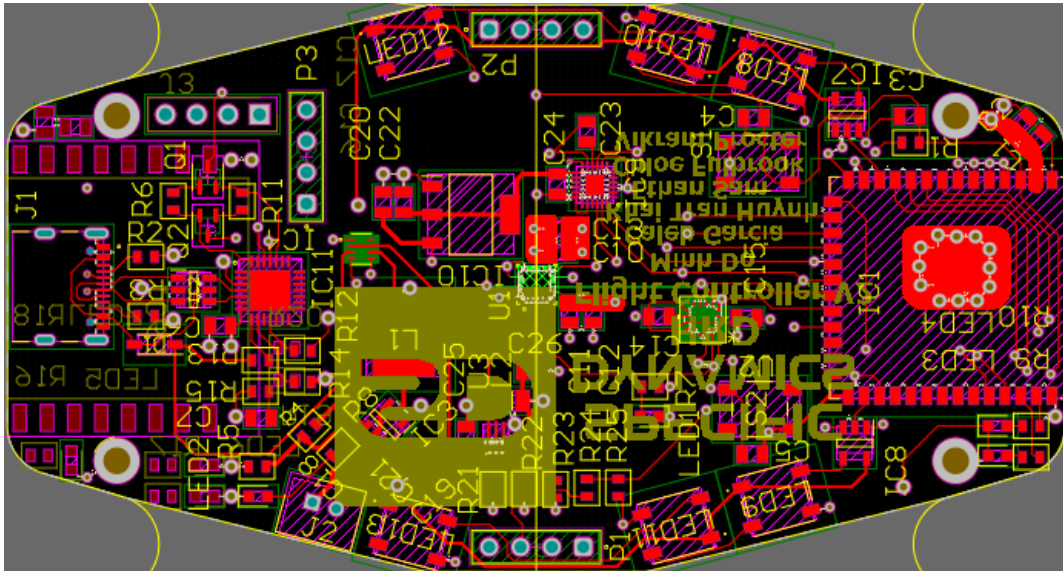


Figure : Flight Controller Top Layer

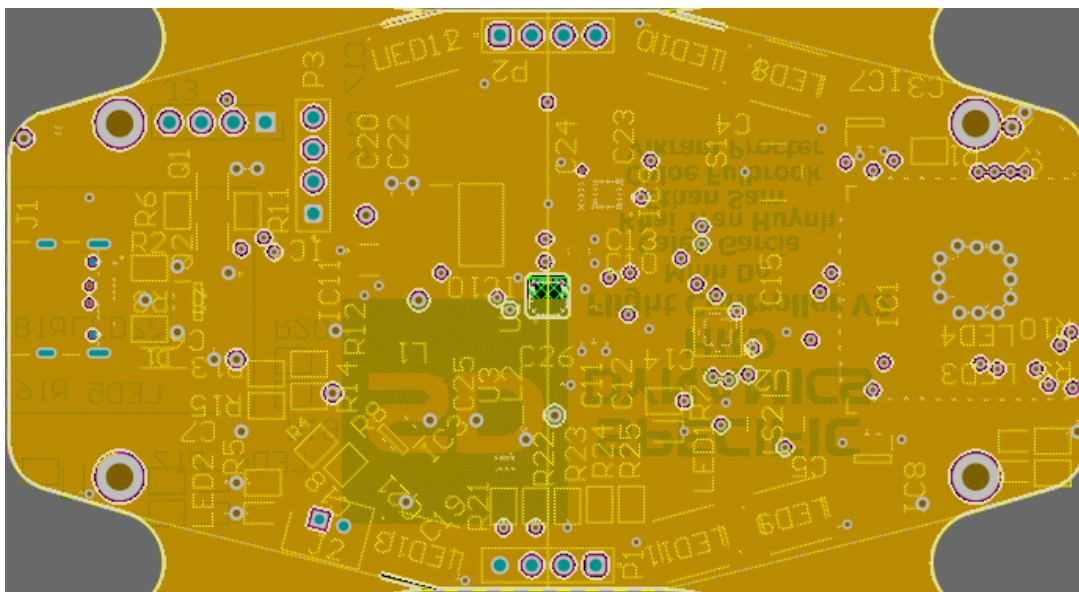


Figure : Flight Controller GND Layer

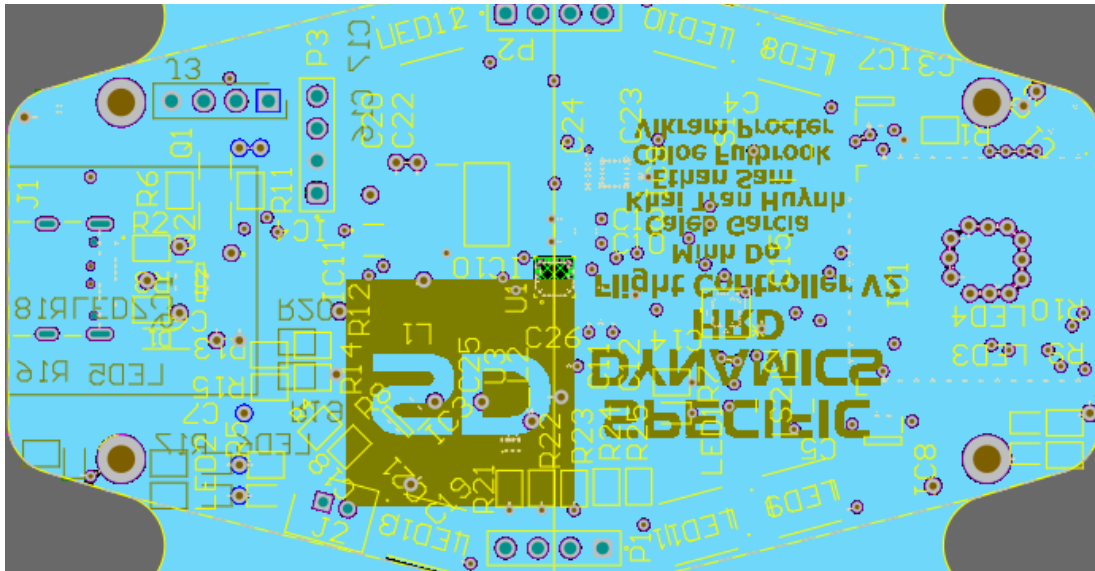


Figure : Flight Controller 5V Layer

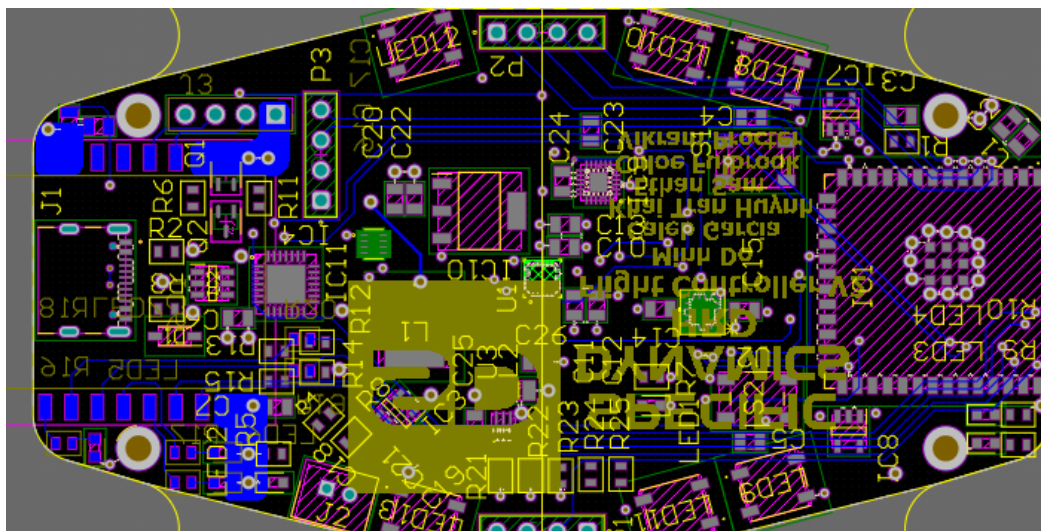


Figure : Flight Controller Bottom Layer

13.3. Remote Control Board Layout

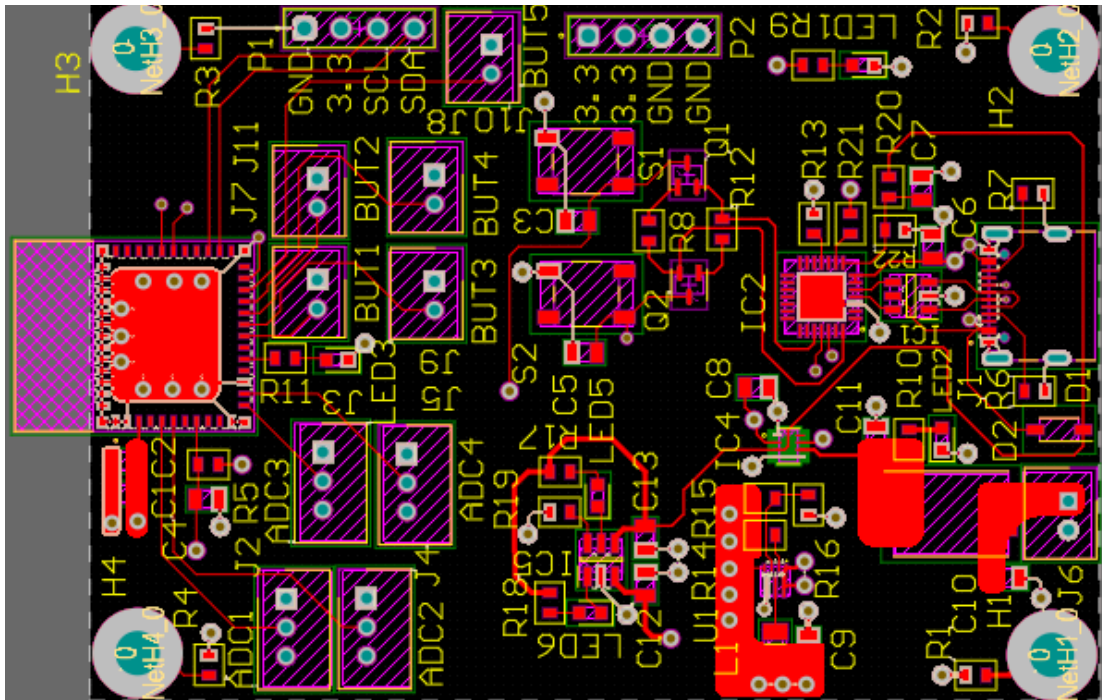


Figure : Controller Top Layer

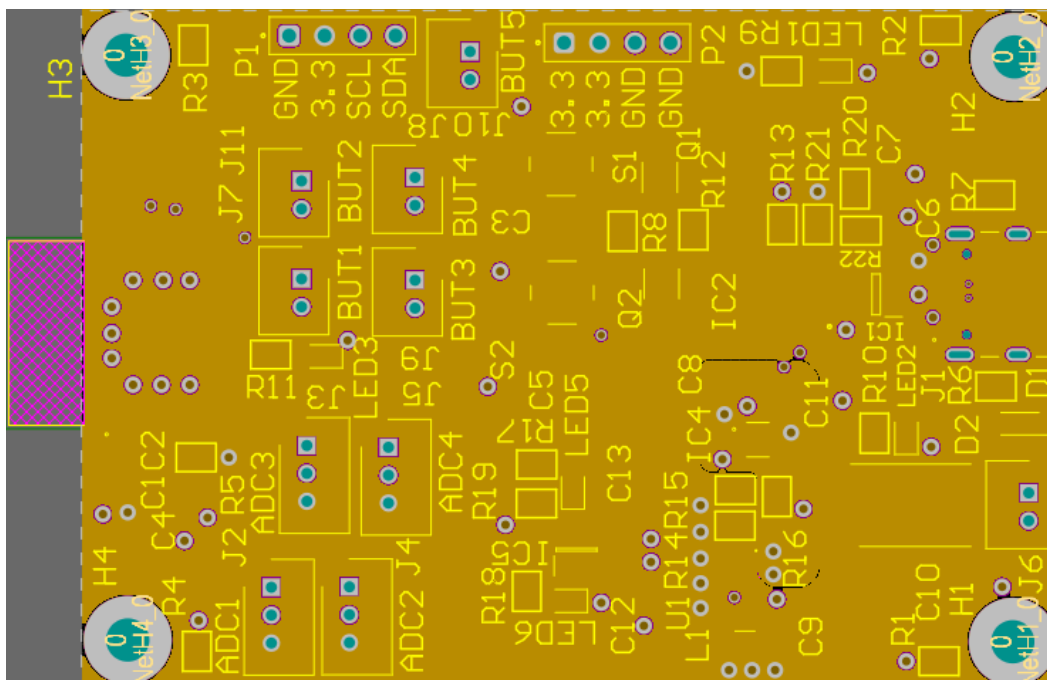


Figure : Controller PWR Layer

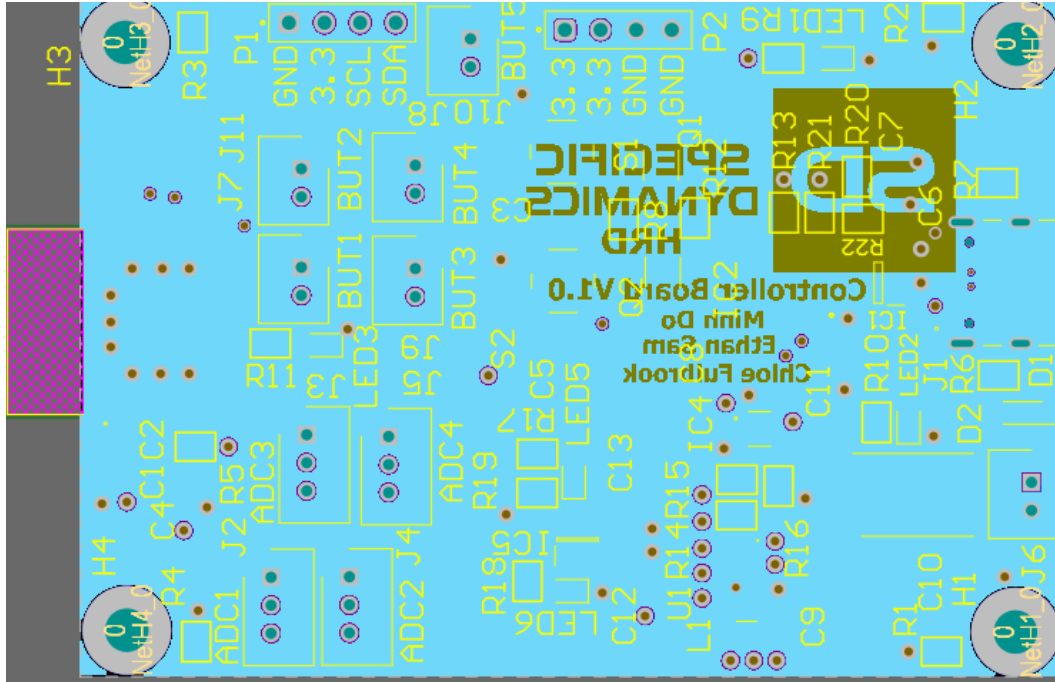


Figure : Controller GND Layer

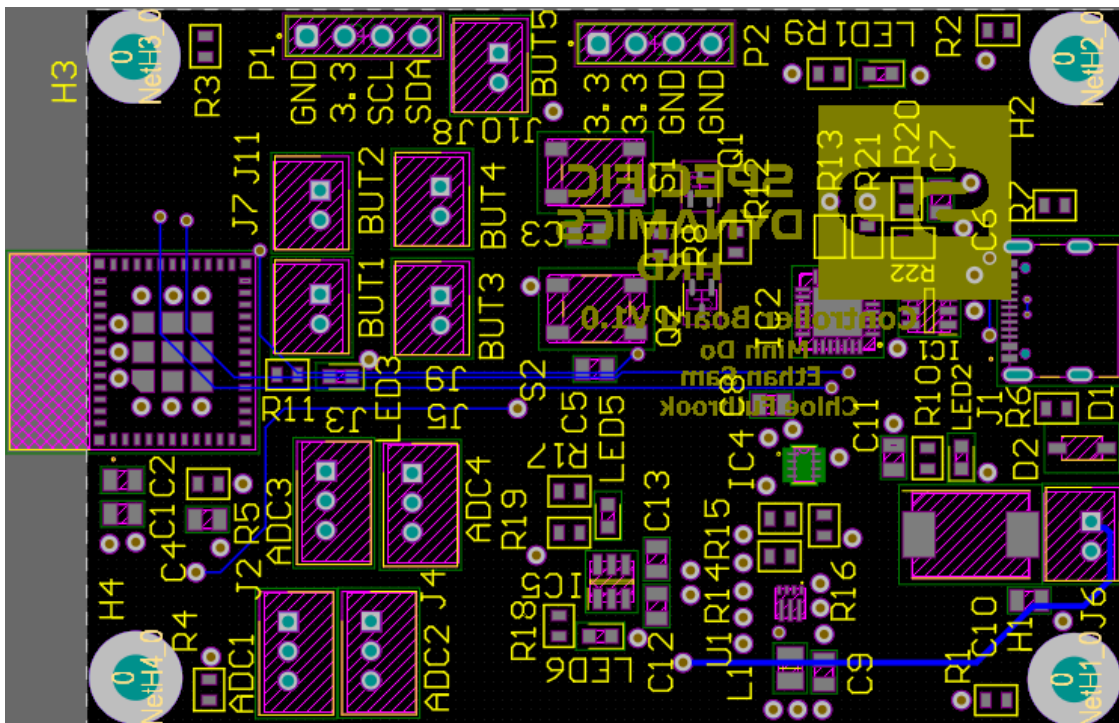


Figure : Controller Bottom Layer

14. Appendix III: Bill of Materials

1. ESC / Airframe Board

RefDes	Part Number	Description	Qty	Unit Cost (CAD)
C1, C2	—	2.2 μ F ceramic capacitor	2	\$0.12
C3	—	4.7 μ F ceramic capacitor	1	\$0.12
C4, C5, C6, C7, C8, C10, C14	—	100 nF ceramic capacitor	7	\$0.12
C9, C12	—	1 μ F ceramic capacitor	2	\$0.12
C11, C13	—	15 pF ceramic capacitor	2	\$0.12
C15, C16, C17	—	10 μ F ceramic capacitor	3	\$0.12
C18, C19, C20	—	22 μ F ceramic capacitor	3	\$0.12
C21A–D, C22A–D, C23A–D, C25A–D, C26A–D, C28A–D	—	330 pF ceramic capacitor	24	\$0.12
C24A–D	—	1 nF ceramic capacitor	4	\$0.12
C27A–D, C31A–D	—	100 nF ceramic capacitor	8	\$0.12
C29A–D, C32A–D, C35A–D	—	1 μ F ceramic capacitor	12	\$0.12
C30A–D	UWT1V221MNL1GS	220 μ F electrolytic capacitor	4	\$0.80

C33A–D	—	10 μ F ceramic capacitor	4	\$0.12
C34A–D	—	470 nF ceramic capacitor	4	\$0.12
D1	150060SS75000	Green LED	1	\$0.23
D2, D3	150060VS75000	Red LED	2	\$0.23
F1	0685P2000-01	2 A fuse	1	\$1.18
J1	S3B-XH-SM4-TB(LF)(S N)	3-pin JST XH connector	1	\$0.89
L1	SPM6530T-2R2M	2.2 μ H power inductor	1	\$1.37
P1, P2	61300411121	4-pin male header	2	\$0.28
P3	61300311121	3-pin male header	1	\$0.19
Q1A–D	MCP6499T-E/SL	Quad op-amp (BEMF sensing)	4	\$1.37
Q2A–D through Q7A–D	CSD17581Q5A	N-channel MOSFET	24	\$1.85
R1, R8	—	165 Ω resistor	2	\$0.16
R2, R4, R7	—	10 k Ω resistor	3	\$0.16
R5	—	400 Ω resistor	1	\$0.16
R6	—	33.3 k Ω resistor	1	\$0.16

R9A–D, R13A–D, R14A–D, R15A–D, R16A–D, R24A–D, R25A–D, R26A–D, R27A–D, R30A–D, R34A–D	—	0 Ω resistor (jumper)	44	\$0.16
R10A–D, R11A–D, R12A–D, R28A–D, R31A–D, R32A–D	—	10 k Ω resistor	24	\$0.16
R17A–D, R18A–D	YC164-JR-072KL	2 k Ω resistor array (1206)	8	\$0.16
R19A–D	—	100 Ω resistor	4	\$0.16
R20A–D, R36	—	DNF (do not fit)	5	—
R21A–D, R22A–D, R23A–D	—	4.3 k Ω resistor	12	\$0.16
R29A–D	—	0.5 m Ω current-sense resistor	4	\$0.16
R33A–D	—	100 k Ω resistor	4	\$0.16
S1–S12	—	Solder pad (motor terminal)	12	—
SW1	PTS647SM38SMTR2LF S	Tactile switch	1	\$0.42
U1	STM32F405RGT6	MCU (ARM Cortex-M4)	1	\$16.15
U2	TPS563209DDCR	3.3 V buck converter	1	\$1.58
U4A–D	DRV8329BREER	Three-phase gate driver	4	\$4.01

Y1	ABM8G-16.000MHZ-18-D2Y-T	16 MHz crystal	1	\$0.67
----	--------------------------	----------------	---	--------

ESC Board Total: \$119.06 | Component count: 210

2. Avionics / PID Board

RefDes	Part Number	Description	Qty	Unit Cost (CAD)
C1, C14, C18	—	10 μ F ceramic capacitor	3	\$0.12
C2, C10, C11	—	100 nF ceramic capacitor	3	\$0.12
C3, C4, C5, C6, C16	—	10 μ F 0603 ceramic capacitor	5	\$0.12
C7, C15	—	1 μ F ceramic capacitor	2	\$0.12
C12, C13	—	2.2 μ F ceramic capacitor	2	\$0.12
C17, C23, C24	—	100 nF 0603 ceramic capacitor	3	\$0.12
C19, C20, C21, C26	—	22 μ F ceramic capacitor	4	\$0.12
C22	—	22 μ F 0603 25 V ceramic capacitor	1	\$0.12
C25	—	10 μ F 0603 25 V ceramic capacitor	1	\$0.12
D1	1N5819HW-7-F	Schottky diode	1	\$0.39
IC1	ESP32-S3-WROOM-1-N16	ESP32-S3 Wi-Fi/BLE module	1	\$9.19

IC2	USBLC6-2SC6	USB ESD protection	1	\$0.52
IC3	TPS563209DDCR	3.3 V buck converter	1	\$1.58
IC4	CP2102N-A02-GQFN28	USB-to-UART bridge	1	\$6.71
IC7, IC8	SN74AHCT1G125DBVR	Single-gate bus buffer (level shifter)	2	\$0.22
IC10	LD1117S18CTR	1.8 V LDO regulator	1	\$0.64
IC11	LM66200DRLR	Ideal diode	1	\$0.82
J1	USB4105-GF-A	USB-C receptacle	1	\$1.14
J2	440054-2	2-pin connector	1	\$0.15
J3	61300411121	4-pin male header	1	\$0.28
L1	VLS6045EX-3R3N	3.3 μ H power inductor	1	\$0.50
L2	74479276147C	Ferrite bead	1	\$1.62
LED1–LED7	LTST-C190GKT	Green LED	7	\$0.22
LED8–LED13	COM-16347	RGB addressable LED	6	\$1.06
P1, P2, P3	61300411121	4-pin male header	3	\$0.28
Q1, Q2	L8050QLT1G	NPN transistor	2	\$0.17
R1, R15, R24, R25	—	47.5 k Ω resistor	4	\$0.16
R2, R3	—	5.1 k Ω resistor	2	\$0.16
R4	—	48.7 k Ω resistor	1	\$0.16

R5, R6, R11, R12	—	10 kΩ resistor	4	\$0.16
R7, R16, R17, R18, R19, R20	—	470 Ω resistor	6	\$0.16
R8	—	1.2 kΩ resistor	1	\$0.16
R9, R10	—	330 Ω resistor	2	\$0.16
R13	—	22.1 kΩ resistor	1	\$0.16
R14	—	1 kΩ resistor	1	\$0.16
R21	—	1 MΩ resistor	1	\$0.16
R22	—	510 kΩ resistor	1	\$0.16
R23	—	162 kΩ resistor	1	\$0.16
S1, S2	PTS647SM38SMTR2LFS	Tactile switch	2	\$0.42
U1	ICM-40609-D	6-axis IMU (accel/gyro)	1	\$6.66
U2	MMC5983MA	Magnetometer	1	\$4.33
U3	TLV62085RLTR	3.3 V buck converter	1	\$1.06

Avionics Board Total: \$52.83 | Component count: 80

3. Controller Board

RefDes	Part Number	Description	Qty	Unit Cost (CAD)
C1, C3, C5, C6	—	10 μF 0603 ceramic capacitor	4	\$0.12
C2	—	100 nF 0603 ceramic capacitor	1	\$0.12

C4	—	1 μ F 0603 ceramic capacitor	1	\$0.12
C7, C13	—	1 μ F ceramic capacitor	2	\$0.12
C8, C10, C11	—	10 μ F 0603 25 V ceramic capacitor	3	\$0.12
C9	—	22 μ F ceramic capacitor	1	\$0.12
C12	—	10 μ F ceramic capacitor	1	\$0.12
D1	1N5819HW-7-F	Schottky diode	1	\$0.39
D2	B540C-13-F	Schottky diode (5 A)	1	\$1.15
IC1	USBLC6-2SC6	USB ESD protection	1	\$0.52
IC2	CP2102N-A02-GQFN28	USB-to-UART bridge	1	\$6.71
IC4	LM66200DRLR	Ideal diode	1	\$0.82
IC5	TP4057-42-SOT26-R	Li-ion battery charger	1	\$0.37
J1	USB4105-GF-A	USB-C receptacle	1	\$1.14
J2, J3, J4, J5	440054-3	3-pin connector	4	\$0.23
J6–J11	440054-2	2-pin connector	6	\$0.15
L1	74479276147C	Ferrite bead	1	\$1.62
LED1–LED3, LED5, LED6	LTST-C190GKT	Green LED	5	\$0.22

P1, P2	61300411121	4-pin male header	2	\$0.28
Q1, Q2	L8050QLT1G	NPN transistor	2	\$0.17
R1, R2, R3, R4	—	0 Ω resistor (jumper)	4	\$0.16
R5, R8, R12, R13	—	10 k Ω resistor	4	\$0.16
R6, R7	—	5.1 k Ω resistor	2	\$0.16
R9	—	470 Ω resistor	1	\$0.16
R10	—	500 Ω resistor	1	\$0.16
R11	—	330 Ω resistor	1	\$0.16
R14	—	1 M Ω resistor	1	\$0.16
R15	—	510 k Ω resistor	1	\$0.16
R16	—	162 k Ω resistor	1	\$0.16
R17, R18	—	600 Ω resistor	2	\$0.16
R19	—	1.6 k Ω resistor	1	\$0.16
R20	—	22.1 k Ω resistor	1	\$0.16
R21	—	1 k Ω resistor	1	\$0.16
R22	—	47.5 k Ω resistor	1	\$0.16
S1, S2	PTS647SM38SMTR2LFS	Tactile switch	2	\$0.42
U1	TLV62085RLTR	3.3 V buck converter	1	\$1.06

U3	ESP32-C3-MINI-1-N4	ESP32-C3 Wi-Fi/BLE module	1	\$4.92
----	--------------------	---------------------------	---	--------

Controller Board Total: \$28.44 | Component count: 61

Summary

Board	Unique Line Items	Total Components	Cost
ESC / Airframe	41	210	\$119.06
Avionics / PID	42	80	\$52.83
Controller	37	61	\$28.44
Total	—	351	\$200.33

15. Appendix IV: Prototype Photograph



Figure : Front and Side View of Drone Prototype

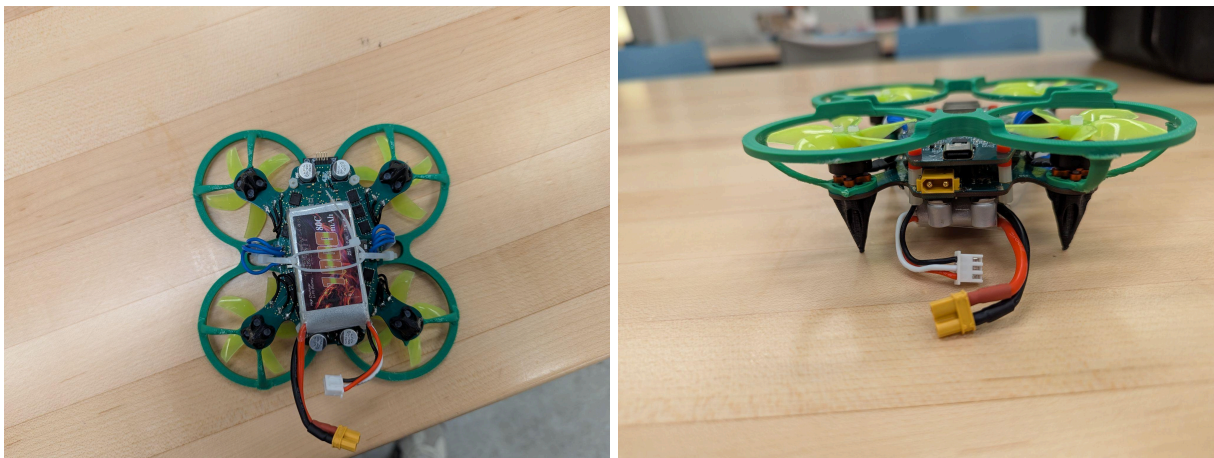


Figure : Bottom and Rear View of Drone Prototype



Figure: Controller Front and Back

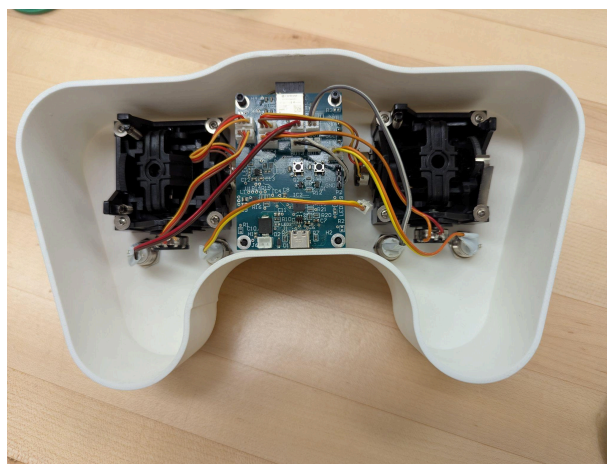
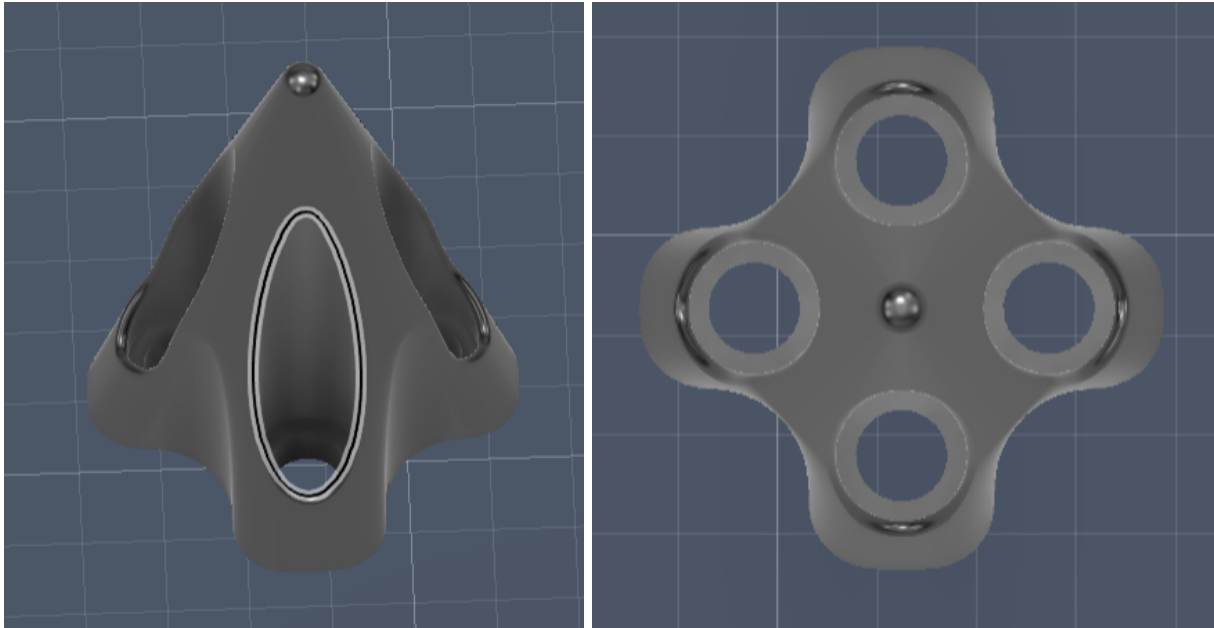


Figure: Wiring for Controller

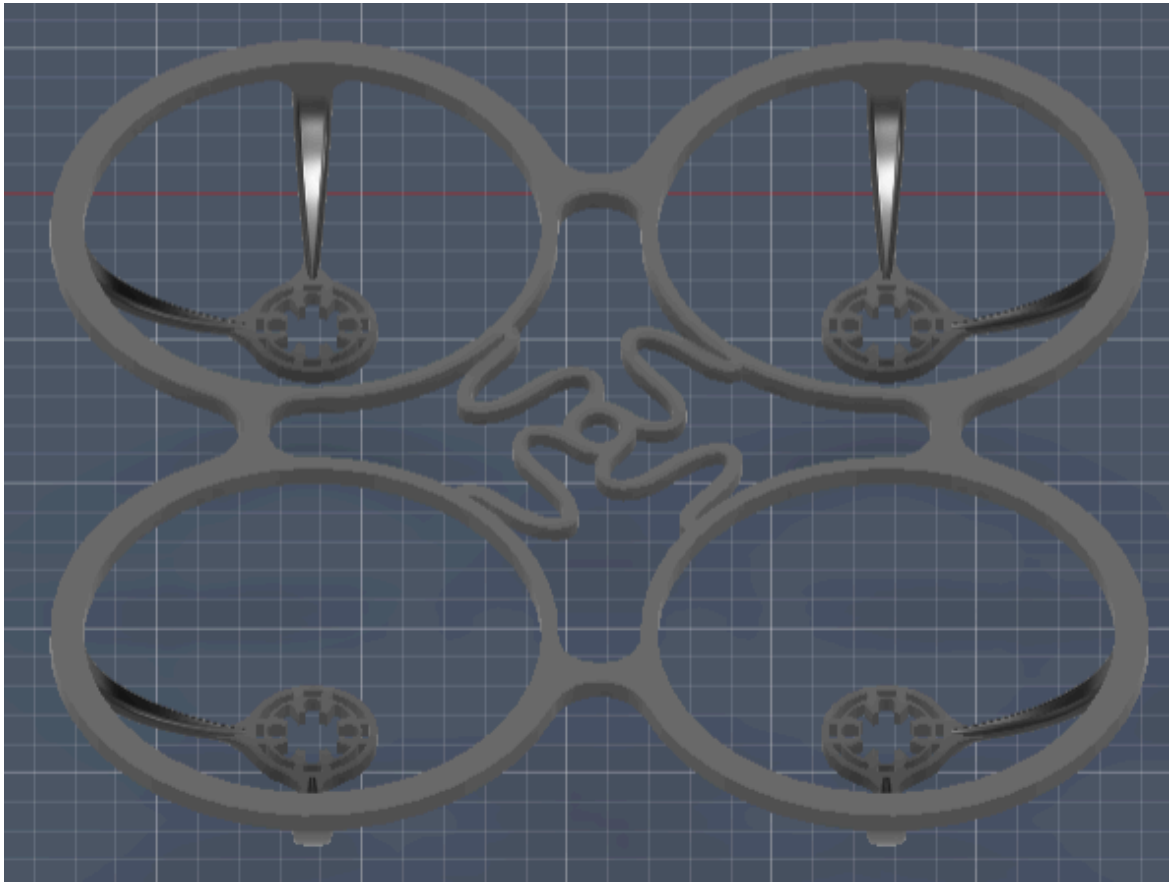
16. Appendix V: 3D Model Drawings

This appendix includes the 3D model drawings created for this project. Drawings were created for the propeller guards and the remote controller case.

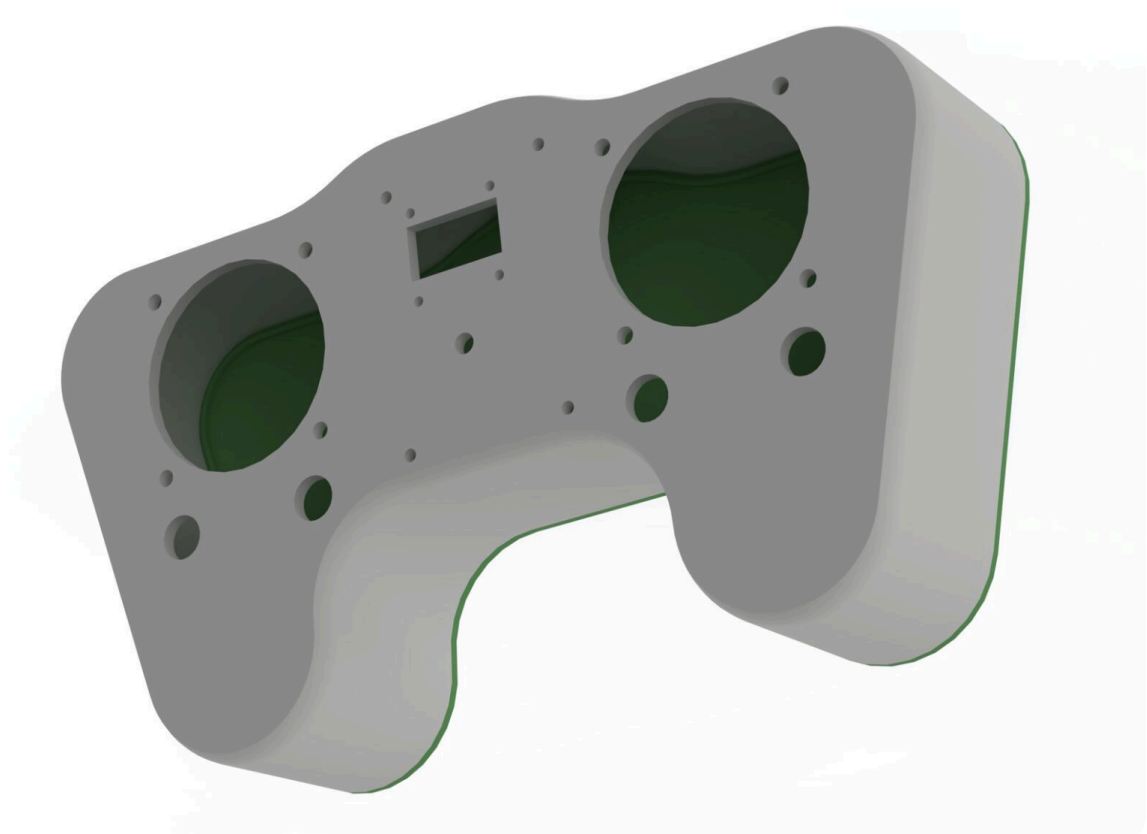
16.1. Drone Landing Leg Drawing



16.2. Propeller Guards Drawing



16.3. Remote Controller Case Drawing



17. Appendix VI: Schedule and Expenditure Variance

Line Item (WBS ID)	Planned (\$)	Actual (\$)	Delta (\$)	Reason for Variance
1.1.1.5 ESC Test Board Assembly	40	60	+20	Two board revisions (\$10) plus stencil (\$10) plus components (\$40); MOSFET orientation error on Rev 1 required respin
1.1.3.1 Single Motor Testing Rig	10	0	-10	Built from recycled materials

1.2.1.4 Avionics Test Board Assembly	40	150	+110	10 boards plus stencil from JLCPCB (\$86) and components for one assembled board (\$63); higher than planned due to ESP32-S3, IMU, and magnetometer component costs
1.2.3.1 Bench Test Rig	50	0	-50	Built from recycled components
1.3.1.1 Source Hardware (Comms/UWB)	30	25	-5	UWB descope; gimbals recycled from IDEAFly MC6 controller
1.3.1.4 Controller Board Assembly	50	44	-6	Five boards from JLCPCB (\$16) plus components (\$28); under budget
1.3.1.7 UWB Anchor Board Assembly	650	0	-650	Descope — UWB subsystem removed prior to final submission
1.3.2.1 ADC Handling (gimbals, dev board)	20	0	-20	Used recycled parts
1.3.2.4 Position Estimate test rig	10	0	-10	Descope with UWB
1.4.1.2 ESC / FC Interface Boards	80	689	+609	Final ESC/airframe boards: 6 PCBAs plus 9 bare boards from JLCPCB (\$540); final flight controller boards (\$86) plus components (\$63). Largest overrun - original PEP significantly underestimated PCBA costs
1.4.3.2 Assembly hardware	30	28	-2	M2 standoffs and screws; on budget
1.4.3.3 Controller mechanical	10	25	+15	Panel-mounted buttons and switches; slightly over due to sourcing from Digi-Key rather than bulk
Unplanned: Motors, propellers, battery	0	~400	+400	COTS flight hardware (YSIDO motors, propellers, LiPo batteries) not itemized in original PEP
Unplanned: Extra components and backups	0	~500	+500	Replacement parts, backup components, and re-orders across multiple Digi-Key and AliExpress orders
Unplanned: Shipping and duties	0	~280	+280	International shipping from JLCPCB, AliExpress, and Digi-Key; import duties on Chinese orders
Total	1,020	~2,200	+1,180	

Milestone	Planned Date (PEP)	Actual Date	Delta (days)	Reason for Variance
Initial Requirements Spec	Feb 23, 2026	Feb 24, 2026	+1	Minor — within grace
Initial Acceptance Test Plan	Feb 23, 2026	Feb 24, 2026	+1	Minor — within grace
Project Execution Plan	Feb 23, 2026	Feb 24, 2026	+1	Minor — within grace
Design Review Presentation	Feb 5, 2026	Feb 5, 2026	0	On time
MVP Demo	Mar 10, 2026	Mar 10, 2026	0	On time
Custom ESC flight integration	~Mar 15–22, 2026	~Mar 28, 2026	+6 to +13	Delayed by ESC test board respin and CNY shipping; integration completed once revised boards arrived and were assembled
UWB multi-anchor ranging	~Mar 2026	Descoped	—	Descoped prior to final submission to prioritize core flight capability
First untethered hover	N/A (not in PEP)	~Apr 3, 2026	—	Not in original PEP; achieved after ESC integration and PID retuning on final hardware
Design Specification	Apr 6, 2026	Apr 6, 2026	0	On time
Updated Requirements Spec	Apr 6, 2026	Apr 6, 2026	0	On time
Updated Acceptance Test Plan	Apr 6, 2026	Apr 6, 2026	0	On time
Acceptance Test Record	Apr 6, 2026	Apr 6, 2026	0	On time

The most significant schedule deviations encountered during the project are discussed below.

The single largest schedule impact was the time required to debug hardware issues, particularly on the custom ESC test board. An error in MOSFET orientation on the initial ESC test board revision required a board respin, which introduced a multi-week delay to the ESC development timeline. This delay was compounded by Chinese New Year shipping delays from JLCPCB, which extended the lead time for the revised

boards. The combination of the design error and the fabrication delay compressed the time available for ESC validation and subsequent flight integration. Assembly and construction of the boards also took longer than anticipated due to the fine-pitch components and the need for careful hand soldering of the power stage. Despite these setbacks, the custom ESC was ultimately integrated into the flying platform and performed well; outperforming the commercial off-the-shelf ESC that had been used as a temporary stand-in during early flight controller development.

A recurring challenge was the requirement for all subsystems to be ready simultaneously before integration testing could begin. The controller and transmitter firmware was completed by the MVP milestone, but could not be integrated until the drone hardware was assembled and ready to receive commands. Similarly, the PID flight control firmware was ready for testing well in advance of the hardware, but early testing on the flight controller development board with a commercial off-the-shelf ESC proved problematic; the off-the-shelf ESC behaved differently from the team's custom motor drive, and the development board used a different IMU than the final avionics board. These discrepancies meant that PID tuning work performed on the dev board did not transfer cleanly to the final platform, and tuning had to be revisited once the custom ESC and final avionics board were integrated. Documentation also consumed more time than planned, particularly the formal reports and the design specification.

The ultra-wideband indoor positioning subsystem was planned from the outset as a parallel development track with the understanding that it would be the first feature descope if the team encountered schedule pressure on the core flight platform. Development on the UWB anchor boards and ranging protocol proceeded in parallel with the main drone build, but as the ESC board revision delay and integration challenges consumed more of the team's available time, the decision was made to formally descope UWB prior to the final submission. This was a deliberate, pre-planned risk mitigation rather than an uncontrolled failure; the team had structured the project so that UWB could be removed cleanly without impacting the core flight capability.

Firmware integration across subsystems proceeded more quickly than expected, attributable to a solid software architecture established early in the project that made it straightforward to slot individual modules (motor control, PID, communications) together. The PID control implementation also progressed rapidly once the team had completed background research into cascaded control architectures and the Mahony filter; the actual coding and initial tuning were completed ahead of schedule. These areas of ahead-of-plan progress partially offset the hardware delays described above.

Beyond the ESC board respin, the team's fabrication timeline was generally impacted by reliance on JLCPCB for all board orders. While JLCPCB offers competitive pricing and quality, the shipping time from China (typically 5 -10 business days) and the Chinese New Year factory shutdown created schedule sensitivity. Future projects of similar scope should account for at least one board respin per critical subsystem and should front-load PCB orders to avoid holiday periods.

The final project spend of approximately \$2,200 represents a \$1,180 overrun against the original PEP budget of \$1,020, but remains under the team's \$2,500 out-of-pocket cap.

The single largest budget variance was the ESC/FC interface boards line item (1.4.1.2), which came in at \$689 against a planned \$80. The original PEP significantly underestimated the cost of the final ESC/airframe board, which required JLCPCB's pick-and-place assembly service for six fully assembled boards plus nine additional bare boards. The final flight controller boards and their components added a further \$149 to this line item. The avionics test board assembly (1.2.1.4) also exceeded its budget by \$110, driven by the component costs of the ESP32-S3 module, ICM-40609-D IMU, and MMC5983MA magnetometer, which were more expensive than the generic estimate in the original plan.

The largest savings came from the UWB descope. The UWB anchor board assembly (1.3.1.7) had been the single largest planned line item at \$650, and descopeing it to \$0 offset a substantial portion of the overruns elsewhere. Several other line items came in under budget or at zero because the team used recycled materials for test rigs and bench setups.

The most significant gap in the original PEP budget was the omission of COTS flight hardware - motors, propellers, and batteries - which were not itemized in the original work breakdown structure. These items, combined with extra component orders for backups and replacements and international shipping and import duties, accounted for approximately \$1,180 in unplanned expenditure. Future projects should include explicit line items for consumable flight hardware, shipping costs, and a contingency margin for backup components, as these categories represented the majority of the budget overrun.

Costs were split roughly evenly across the six team members, with a final per-member share of approximately \$367, well within the \$417 per-member cap.

VI.5 Lessons Learned

The MOSFET orientation error on the ESC test board was a preventable mistake that cost multiple weeks. Future projects should plan for at least one board respin per critical PCB, scheduling fabrication timelines accordingly rather than treating the first order as the final board.

The team's reliance on JLCPCB for all board fabrication meant that the Chinese New Year factory shutdown directly impacted the project schedule. Orders should be placed well in advance of known shutdown periods, and local rapid-prototyping options such as AP Circuits in Calgary should be identified as contingency sources even if they are more expensive.

The team's decision to establish a clean firmware architecture with well-defined interfaces between modules (motor control, PID, communications) early in the project was one of the strongest contributors to schedule recovery. When integration began, the software components slotted together with minimal rework. Teams should resist the temptation to skip architectural planning in favor of immediately writing code, as investing in software architecture early pays off significantly during integration.

PID tuning performed on a development board with a commercial ESC and a different IMU did not transfer cleanly to the final platform with the custom ESC and production avionics board. While early dev-board testing was valuable for validating the control architecture, the gains had to be revisited once the final hardware was assembled. Where possible, subsystem testing should be performed on the actual target hardware, even if it means waiting for boards to arrive rather than testing on a surrogate.

The decision to structure UWB as a parallel, independently removable work stream (rather than deeply integrating it into the core architecture) meant that descopeing it did not disrupt the flight platform. This approach should be replicated for any feature that carries schedule risk: design the interfaces so that the feature can be removed without breaking the system.

Consistent version control workflows - individual branches, clear commit messages, and regular merges - were essential for keeping six contributors synchronized across firmware, schematics, and documentation without overwriting each other's work. Good Git management proved crucial for a team of this size.

With seven different boards sharing overlapping component libraries, the team encountered double-ordering, under-ordering, misplaced parts, and confusion over which components belonged to which board. Future projects with multiple PCBs should

maintain a single consolidated BOM tracker and designate one person to manage procurement across all boards.

The team's approach of maintaining backup components, backup boards, and backup subassemblies proved invaluable when boards arrived with errors, components were damaged during hand soldering, or parts were lost during assembly. Ordering significantly more spare components and PCBs than the minimum required is one of the highest-return investments a hardware project can make, as the marginal cost of extra components is negligible compared to the schedule cost of waiting for a replacement order.

Formal reports, design specifications, and test records consistently consumed more time than planned. Teams should allocate dedicated documentation time in the schedule rather than treating it as an afterthought that can be completed in the final week.

